Final Project – Integrated Dependability Engineering Capstone

Fault-Tolerant University Records Service: Gateway, Observability, Security and Fault Injection, Baseline versus Improved

Astana IT University | Fault Tolerance and Dependable Computing | Academic Year 2026–2027

Student: Shamil Yakupov, ID 255193 | Instructor: Azamat Serek | Individual work

Experiments run on 6 November 2026 | Report date: 6 November 2026

Context note: all data in this project are synthetic. The system under test is a local Kubernetes lab; no real users, production systems or real university infrastructure were involved. The application is the hoWallet Go API, used (as in the midterm and endterm) only as a stand-in for a synthetic university service.

Contribution: this is individual work with no other team members. Shamil Yakupov is responsible for all of it: requirements, design, configuration, scripts, experiments, analysis and this report.

1. Abstract and Keywords

This project evaluates a dependable records service on a one-node Kubernetes lab, by comparing an improved configuration (nginx gateway with bounded retries, two API replicas, probes, drain on shutdown, three Prometheus alerts) with a retained baseline (one replica, no gateway). Both share one PostgreSQL and the same application code. Eight experiments from the assignment matrix were run with 2–4 repetitions per scenario under a test plan written beforehand. Losing one API instance cost the baseline a 25.0–25.6 s outage after a graceful delete and 2.0 s after a forced one, and cost the improved configuration no failed request in 418. Under a 2 s delay on one replica's database link, the improved configuration kept all 247 requests successful, each bounded near 5 s, while the baseline lost 30 % with a maximum of 34 s. No acknowledged write was lost or duplicated in any run of either configuration; that result belongs to the application, so it does not separate the two. The comparison is not uniformly favourable: during a database outage the baseline answered more requests successfully (37 % vs 8 %); the gain there is bounded response time, not availability. The load result is partly an artefact of a per-process rate limiter, one access-control case is recorded as failed because its expected value was wrong, the rollback experiment shows no gain from the improved configuration, and PostgreSQL remains a single point of failure. Ranges describe behaviour on one machine and are not statistical estimates.

Keywords: dependability, fault injection, availability, MTTR, bounded retries, idempotency, transaction rollback, Kubernetes, nginx, Prometheus, observability, access control.

2. Introduction, Motivation and System Context

A records service for a university has a simple promise: a student's payment or registration entry is recorded exactly once, can be read back, and is visible only to the people allowed to see it. A short outage near a deadline is a nuisance; a fee charged twice or a payment lost is worse. The work therefore asks a narrow question: which failures does the improved configuration actually tolerate, at what cost, and what can the evidence not show? The assignment's dependability vocabulary and means follow Avizienis et al. [1].

System. Option A of the assignment: the project continues the university information system from the midterm and endterm. The hoWallet Go API, written by the author as a personal-finance tracker, keeps the role of the university **Registration/Records** service, and PostgreSQL its data store. All accounts, households and transactions are synthetic lab records. The domain words in logs and tables come from the code: a *household* is the tenant (one student's record space), an *account* is a ledger, and a *transaction* is a record write (for example a fee payment). The

system has more than three independently identifiable services: gateway, API, database, Prometheus, Grafana and a migration job, plus API modules (authentication, transactions, background jobs). The critical multi-step workflow is posting a record, which inserts a row and moves a ledger balance in one database transaction.

Relation to the endterm. The manifests and scripts continue the endterm lab. Three things changed. The configuration label `endterm` is now `improved`. The gateway log also records `$upstream_status`, so retries can be counted; no timeout or retry value changed. The PostgreSQL memory limit went from 256 Mi to 1 Gi after an OOM kill (Section 9.5). Three experiments are new (Exp 3 network delay, Exp 4 interrupted transaction, Exp 8 rollback), and all eight were run again, so endterm figures are used only where stated.

Correction to the endterm. The endterm reported a 23 s baseline outage after a replica loss. That figure was understated: its probe stopped while requests were still timing out, so it was a lower bound. Exp 1 now waits for 14.5 s of clean answers and measures **25.0–25.6 s**. Section 10.3 gives the detail.

3. Requirements, Assumptions and Dependability Objectives

3.1 Users, workflow, data and consequences of failure

Table 1. Users, workflow, data and consequences of failure (`notes/requirements.md`).

Item	Description
Users	Students (own record space), staff/members invited into a record space, operators (cluster admin)
Main workflow	Sign in → choose record space (`X-Household-ID`) → read ledgers and records → post a record (fee payment) that moves a ledger balance
Data handled	Synthetic users (e-mail, bcrypt hash), record spaces, ledgers with balances, records (amount, currency, date, `client_id`), request/audit logs
Consequence of failure	Unavailable: students cannot see or pay fees (deadline risk). **Wrong or duplicated record: a fee charged twice or a payment lost – worse than an outage** (repo rule: "a wrong number is worse than an honest error"). Disclosure: one student sees another's records

3.2 Critical functions

Table 2. Critical functions.

ID	Critical function	Components on the path	Multi-step?
CF1	**Authenticate** (log in, refresh, verify token on every call)	gateway → api (auth service, session check in Postgres) → postgres	yes: credential check + session row + token issue
CF2	**Post a record and update the ledger balance**	gateway → api (membership check, money resolution, DB transaction: insert row + update balance, then audit) → postgres	**yes** – insert and balance update in one DB transaction (`internal/service/transaction.go:501-536`), idempotent by `client_id`
CF3	**Read records and balances**	gateway → api (JWT, live session, membership) → postgres	no (read-only, safe to retry)
CF4	**Tenant isolation** (only members see/change a record space)	api → postgres	– (cross-cutting)
CF5 (supporting)	Observe and alert	prometheus → api pods; grafana → prometheus	–

3.3 Service-level objectives

A request **fails** if it gets a 5xx, a timeout or a connection error. A 429 is the rate limiter working as designed and is counted separately. The structure of the objectives follows [2]. Table 39 in Section 10.11 gives the measured outcome per objective.

Table 3. Service-level objectives with the improved configuration as the target.

ID	Objective	Metric	Target (improved config)	Window	Checked in
SLO-1	Availability of CF3 in steady state	request-success ratio `ok/(ok+failed)`	≥ 99.5 %	120 s quiet window at 5 req/s	Exp 5 (warm phase), Exp 6 (quiet window)
SLO-2	Availability of CF3 during a single API-instance failure	request-success ratio	≥ 99 % (target 100 %)	injection −5 s … +20 s	Exp 1
SLO-3	Latency of CF3	p95 latency	≤ 500 ms at 30 req/s	60 s load step	Exp 5
SLO-4	Bounded failure latency under any dependency fault	max latency of any answer	GET ≤ 10.5 s (2 tries × `proxy_read_timeout 5s` + margin; endterm E2 measured a 10.0 s maximum), POST ≤ 5.5 s (one try); no hung connection	fault window	Exp 2, Exp 3
SLO-5	Recovery time (RTO)	Exp 1: injection to replacement ready and no failures. Exp 2: restore command to end of the last failed request. Exp 8: undo to bad pod gone	≤ 10 s instance; ≤ 30 s database	per injection	Exp 1, Exp 2, Exp 8
SLO-6	Write integrity of CF2	lost acknowledged writes; duplicate `(household_id, client_id)`; `balance = −Σ amounts`	**0 lost, 0 duplicates, balance check passes**	whole run	Exp 2, Exp 4, Exp 8
SLO-7	Detection time	fault onset → alert `firing`	≤ 30 s; 0 alerts in a quiet window (no false positive)	per injection	Exp 6
SLO-8	Access control	share of unauthenticated / foreign-tenant requests denied; data changed by them	100 % denied (401/404), 0 rows changed, each denial logged with `request_id`	per test case	Exp 7
SLO-9	Backup recoverability (RPO)	records missing after restore vs. records at dump time	0 records lost since the last `pg_dump`; RPO = time since that dump	restore drill	**Not tested in the final project.** Exp 8 as run is a release rollback. The only drill is the earlier one in `ftr-a3`

3.4 Assumptions

- Single-node kind cluster on Docker Desktop/WSL2; node, zone and host failures are **out of scope**.
- One PostgreSQL instance (StatefulSet, 1 replica, PVC 1 Gi). No streaming replica: database failover is not claimed; protection is restart-in-place plus logical backup (`pg_dump`).
- Fault model is one fault at a time (single-fault assumption); faults are injected deliberately by `kubectl`, `docker exec`, SQL locks or config changes.
- Clients reach the cluster through NodePorts; kind SNATs them to one address, so per-IP limits are shared by all test clients.
- Clients retry writes **with the same client_id** (as the real web client's outbox does); the gateway never replays POST.
- The exchange-rate provider is a non-critical external dependency; records in the base currency must not depend on it.
- All data and secrets are synthetic lab placeholders; tests run only in this authorised local environment.
- Clocks of client and cluster are the same host clock (timestamps comparable to ~1 ms).

4. Fault Model, Threat Model and Failure Analysis

4.1 Terminology and fault model

Table 4. Fault, error and failure as used here, after Avizienis et al. [1].

Term	Definition (2004 taxonomy)	Example in this system
Fault	The adjudged or hypothesised cause of an error. Active when it produces an error, dormant otherwise	API pod killed; Postgres stopped; 3 s injected delay; a client sending a forged token; code path that writes a rate of 1 when a rate is missing (design fault)

Term	Definition (2004 taxonomy)	Example in this system
Error	The part of the system state that may lead to a failure	gateway upstream list still contains a dead pod; pgx pool connections broken; a transaction row inserted but balance not yet updated; a duplicate row from a replayed POST
Failure	The delivered service deviates from correct service (observable at the service interface)	client gets 5xx/timeout; a fee shown twice; a balance that differs from the sum of records; another tenant's data returned

Table 5. Fault model.

Class (2004)	Fault	Persistence	Injected in
Physical / operational, crash	API process or pod lost (delete, SIGKILL)	transient (replaced)	Exp 1, Exp 4
Operational, hang	API process frozen (SIGSTOP) – stops answering but keeps TCP	transient (liveness restart)	Exp 6
Dependency, crash/hang	PostgreSQL stopped (scale 0) or frozen	transient (restored)	Exp 2
Interaction, timing	2000 ms delay on the API → Postgres link, beyond the gateway read timeout; table locks	transient	Exp 3, Exp 6
Interaction, overload	traffic step 5 → 30 → 5 req/s	transient	Exp 5
Interaction, malicious / invalid input	missing/forged token, foreign `X-Household-ID`, malformed body, brute force	permanent while attacker acts	Exp 7
Development / configuration	bad release rolled out (`API_PORT=9999`)	permanent until rolled back	Exp 8
Physical, storage	lost database volume	permanent until restored	not injected in the final project

4.2 Failure scenarios and impact

Severity: **H** = data integrity or confidentiality at risk, **M** = critical function unavailable, **L** = degraded but correct.

Table 6. Failure scenarios and failure-impact analysis.

#	Scenario (Exp)	Fault	Error	Failure (if not tolerated)	Impact (sev.)	Detection	Mitigation (improved config)
FS1	Instance failure (Exp 1)	one API pod deleted or crashes	gateway/kube-proxy still route to the dead endpoint for ≤ 2 s; in-flight requests cut	5xx / connection reset; baseline: 25.0–25.6 s outage after a graceful delete, 2.0 s after a forced one (Exp 1; the endterm's 23 s was understated)	CF1–CF3 unavailable (M)	readiness probe (2 s), endpoint drop, `ApiReplicaDown` (counts healthy replicas), error-rate alert	2 replicas, `maxUnavailable 0`, preStop drain 5 s, gateway bounded retry on the other pod (`proxy_next_upstream … tries 2`), passive ejection `max_fails=1 fail_timeout=2s`
FS2	Database unavailable (Exp 2)	Postgres scaled to 0 or frozen	pool connections fail; membership/session queries fail	baseline: requests hang until client timeout (12 s); risk of half-applied write	CF1–CF3 down (M); integrity risk (H)	`ApiHighErrorRate`, `ApiHighLatencyP95`, ERROR log "failed to check household membership"	fail fast: gateway 5 s read timeout → JSON 503 + `Retry-After`; membership check fails **closed** (500); write is one DB transaction → nothing half-applied; client retries with same `client_id`
FS3	Network delay / timeout (Exp 3)	`tc netem delay 2000ms` on egress to Postgres in one API pod (the only pod for baseline)	requests exceed their deadline; upstream slots held	unbounded waits; retry storm amplifying load	latency SLO broken (M)	`ApiHighLatencyP95`, gateway log `upstream_response_time`	`proxy_connect_timeout 1s`, `proxy_read_timeout 5s`; **retries bounded to 2 tries** and only for idempotent methods; API server timeouts (read 15 s, write 15 s, header 5 s, `cmd/api/main.go:81-88`)
FS4	Interrupted multi-step transaction (Exp 4)	API process SIGKILLed between "insert record" and "update balance". A loss after commit but before the client saw the answer is a further risk, handled by the same key, and not injected here	uncommitted DB transaction; client does not know whether the write happened	**lost** record (client gives up) or **duplicate** record (client retries a committed write); balance ≠ Σ records	fee lost or charged twice (H)	integrity check: rows vs. acknowledged writes, duplicate `client_id`, balance = −Σ	`RunInTx` (insert + balance in one transaction, rollback on any error), idempotency key `client_id` + unique index → retry returns the first row (200)
FS5	Load step (Exp 5)	offered load 5 → 30 → 5 req/s	queueing in pool (40 conns/replica), CPU throttling, limiter counters fill	p95 above SLO; 429 for everyone behind one IP; 5xx	slow/denied service (M/L)	p95 alert, `http_requests_in_flight`, `db_pool_acquired_conns`	2 replicas share load; per-user/per-IP rate limits answer JSON 429 + `Retry-After` instead of collapsing

#	Scenario (Exp)	Fault	Error	Failure (if not tolerated)	Impact (sev.)	Detection	Mitigation (improved config)
FS6	Monitoring / alert (Exp 6)	a replica hangs (SIGSTOP), the database stops for 25 s, or lock bursts slow it for about 40 s	the system degrades without users reporting it	incident unnoticed (false negative) or alert noise (false positive)	prolonged outage (M)	3 Prometheus rules (for 6–10 s), Grafana dashboard, correlated `request_id` in gateway + API logs	rule `ApiReplicaDown` fixed to count healthy replicas (endterm finding); alerts scoped to `job="api"`, `route!="/health"`
FS7	Unauthorized / invalid operation (Exp 7)	request without token, forged JWT, user B targets A's record space, malformed or huge body, password brute force	request reaches business logic with wrong identity or bad input	disclosure or tampering of another student's records; crash on bad input	confidentiality/integrity (H)	401/403/404/400/413 /429 in logs with `request_id`, `user_id`; `app_logs` rows	JWT (HMAC, `iss`, `exp`), live-session check, verified-email gate, membership check → 404, input validation, 1 MB decode / 2 MB gateway cap, login lockout, rate limits (see `security-controls.md`)
FS8	Recovery / rollback (Exp 8)	release listening on the wrong port (`API_PORT=9999`)	bad pod never ready; rollout stalls	service down after a deploy, if old pods were removed	M	readiness failure (2.1–2.4 s); **no alert watches rollouts**, measured as a monitoring gap	rolling update keeps old pods until new ones are ready; `kubectl rollout undo`. A default RollingUpdate with 1 replica does the same. Database loss is not part of Exp 8: `pg_dump` + `pg_restore` was drilled only in ftr-a3 (19.2 s)

4.3 Threat model

The STRIDE analysis [16] with code references is in `notes/threat-model.md`. Assets: records and balances, credentials, the JWT signing secret, tenant membership and the audit trail. Actors: A1 anonymous attacker, A2 authenticated user of another tenant, A3 compromised pod, A4 abusive or buggy client.

Table 7. Threats per entry point, controls and residual risk (condensed from `notes/threat-model.md`).

Entry point / actor	Threat (STRIDE)	Control	Residual risk
/auth/* (A1)	S: brute force, credential stuffing	10 req/min per IP on /auth; lockout after 5 failures per (e-mail, IP) for 15 min; bcrypt	counters live in each replica's memory (Exp 7: first 429 at attempt 11, not 6)
/auth/* (A1)	S/T: forged JWT	HMAC only, issuer and expiry required; same 401 for all failures	whoever holds JWT_SECRET can mint tokens (A3)
Household API (A2)	I/T: reading or changing another tenant through X-Household-ID or foreign ids	membership check → 404; queries scoped by household_id	tested in Exp 7
Household API (A4)	T: duplicate writes on retry	client_id idempotency key + unique index	only POST /api/transactions has it
Household API (A4)	D: flooding, huge bodies	600/min per user, 1200/min per IP, 60/min creates; 2 MB at the gateway, 1 MB decode	per-replica limits; no statement_timeout
Gateway (A1)	S: spoofed X-Real-IP to dodge limits	gateway overwrites X-Real-IP, clears True-Client-IP	the baseline, with no gateway, honours the header (Exp 7)
Any pod (A3)	E/I: lateral movement	NetworkPolicies, non-root, read-only root filesystem	API database user owns all tables; logs in the same database; not tested in Exp 7
All (A1–A4)	R: denying actions	JSON request log with request_id and user_id; audit_logs; app_logs	no dedicated failed-login event; audit queue drops under load

Table 8. Misuse scenarios tested in Exp 7.

ID	Misuse scenario	Actor	Threat (STRIDE)	Expected control	Evidence
MU1	Request with no token, a garbage token or a forged token	A1	Spoofing	401, nothing changed	status, log line with `request_id`
MU2	User B targets A's record space (read, create, delete) with `X-Household-ID: A`	A2	Information disclosure, tampering	404 `household not found`, A's data unchanged	before/after row snapshot, log lines
MU3	Malformed, negative or oversized input	A4	Tampering, denial of service	400, JSON 413 from the gateway, service stays healthy	status, parallel probe
MU4	Password brute force	A1	Spoofing	401 then 429 with `Retry-After`	status sequence, request log

5. Reliability Engineering Calculations

All values in Sections 5.1–5.3 are **measured-final**: computed by `python analysis/reliability.py` from `evidence/results/reliability-inputs.json` and read from `analysis/reliability-computed.json` when this report is built. Section 5.4 is **illustrative** and is not a measurement.

5.1 Formulas, units and windows

Table 9. Symbols.

Symbol	Meaning	Unit
`T_obs`	observation window length	s
`N_f`	failure episodes observed in the window	count
`D`	total downtime: sum of the lengths of the failure intervals	s
`U = T_obs − D`	total up time	s
`ok`, `failed`	requests answered correctly / failed in the window	count

Table 10. Quantities.

Quantity	Formula	Unit	Meaning
MTTR	`D / N_f`	s	mean time to restore client-visible service
MTTF	`U / N_f`	s	mean up time before a failure
MTBF	`MTTF + MTTR = T_obs / N_f`	s	mean time between the starts of successive failures
Availability	`A = MTTF / (MTTF + MTTR) = U / T_obs`	dimensionless, %	share of the window in which service was up
Observed failure rate	`λ = N_f / T_obs`	1/s (printed as 1/h)	failure episodes per unit time, under the conditions of the window
Request-success ratio	`R_req = ok / (ok + failed)`	dimensionless, %	share of requests served correctly

Repetitions are pooled: `T_obs`, `N_f`, `D`, `ok` and `failed` are summed, then the formulas are applied. With `N_f = 0` the window is right-censored: MTTF is reported as `> T_obs`, MTTR and MTBF as undefined, `A = 1`. Zero failures in a short window does not show that λ is zero; the rule of three gives a rough 95 % upper bound of `λ < 3 / T_obs` for a constant-rate process [4].

- **Window:** from the first client request to the end of the last one in the run, including the time before the fault. Each window encloses one injected fault.
- **Failure interval:** from the start of the first failed request to the end of the last failed request of an episode (client-visible downtime). Failed requests within 2.0 s are merged. Resolution is limited by the probe rate, and the length of a failed request, including the client's timeout, counts towards `D`.
- **Observed failure rate basis:** `λ = N_f / T_obs`, under the conditions of the window.
- **MTTF, MTBF and λ here mostly reflect injection timing, not system reliability.** The experimenter chooses when the fault happens, so MTTF is largely the lead time before the injection and λ is close to `1 / T_obs`. They are computed because the assignment asks for them, and must not be extrapolated to a field value (`notes/reliability.md` §3). The comparable quantities are MTTR, `D`, `R_req`, and `A` between runs with equal windows and fault duration.

5.2 Measured results

Table 11. Measured-final reliability quantities, pooled per experiment and configuration. λ is per hour of window and conditional on injected faults.

Exp	Scenario	Config	Reps	T_obs (s)	N_f	D (s)	MTTR (s)	MTTF (s)	MTBF (s)	A	λ (1/h)	ok / failed	R_req
1	Instance failure	baseline	3	119.5	3	52.6	17.6	22.3	39.8	55.95 %	90	657 / 302	68.51 %
1	Instance failure	improved	3	66.7	0	0.0	undefined	> 66.7, censored	undefined	100.00 %	0	536 / 0	100.00 %
2	Database unavailable	baseline	4	207.0	4	102.7	25.7	26.1	51.8	50.42 %	70	938 / 308	75.28 %

Exp	Scenario	Config	Reps	T_obs (s)	N_f	D (s)	MTTR (s)	MTTF (s)	MTBF (s)	A	λ (1/h)	ok / failed	R_req
2	Database unavailable	improved	4	207.7	4	114.4	28.6	23.3	51.9	44.95 %	69	632 / 618	50.56 %
3	Network delay	baseline	3	268.0	3	101.5	33.8	55.5	89.3	62.13 %	40	465 / 74	86.27 %
3	Network delay	improved	3	247.5	0	0.0	undefined	> 247.5, censored	undefined	100.00 %	0	498 / 0	100.00 %
4	Interrupted transaction	baseline	3	121.5	3	54.8	18.3	22.3	40.5	54.94 %	89	366 / 123	74.85 %
4	Interrupted transaction	improved	3	115.6	0	0.0	undefined	> 115.6, censored	undefined	100.00 %	0	465 / 0	100.00 %
5	Load step	baseline	2	240.4	6	6.2	1.0	39.0	40.1	97.42 %	90	3077 / 41	98.69 %
5	Load step	improved	2	240.4	0	0.0	undefined	> 240.4, censored	undefined	100.00 %	0	4204 / 0	100.00 %
6	Monitoring, pooled	improved	9	720.2	5	81.2	16.2	127.8	144.1	88.72 %	25	3736 / 352	91.39 %
8	Rollback	baseline	2	146.8	0	0.0	undefined	> 146.8, censored	undefined	100.00 %	0	736 / 0	100.00 %
8	Rollback	improved	2	156.0	0	0.0	undefined	> 156.0, censored	undefined	100.00 %	0	782 / 0	100.00 %
7	Unauthorized operation	–	–	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a	reported in 10.9

5.3 Worked calculation and interpretation

Exp 1 baseline, repetition 1 (`exp1-baseline-rep1-graceful`): `T_obs` = 46.13 s, `N_f` = 1, `D` = 25.63 s, ok = 221, `failed` = 149.

```
U       = 46.13 - 25.63        = 20.50 s
MTTR    = 25.63 / 1            = 25.63 s
MTTF    = 20.50 / 1            = 20.50 s   (time around the outage, not a lifetime)
MTBF    = 20.50 + 25.63        = 46.13 s
A       = 20.50 / 46.13        = 44.44 %
lambda  = 1 / 46.13 s          = 78 per hour  (one injected fault in this window)
R_req   = 221 / (221 + 149)      = 59.73 %
```

- **Exp 1.** The pooled MTTR of 17.6 s describes neither behaviour: graceful delete 25.6 s and 25.0 s, forced delete 2.0 s. Improved has no failure episode, so only a lower bound of 66.7 s on MTTF can be stated (rule of three: λ < 162 per hour, about this short window).
- **Exp 2 favours the baseline** on `A` (50.42 % vs 44.95 %) and `R_req` (75.28 % vs 50.56 %). Section 10.4 explains why this is not an availability gain for the baseline and not one for the improved configuration either.
- **Exp 3.** Slow successful requests are not downtime in this calculation. Improved has `D` = 0, yet its p95 is about 5 s, so read this row with its latency.
- **Exp 5.** 429 is excluded from `R_req`, so the baseline's 98.69 % does not show that 1086 requests were refused in the whole window.
- **Exp 6** pools four different scenarios and has no baseline; the scenario rows of Table 27 are the useful ones.

5.4 Theoretical estimate (illustrative, not measured)

Every value in this table is illustrative. The per-instance availability of 0.999 is an assumption chosen to show the shape of the calculation. It was not measured and must not be cited as a result. Assuming independent failures, `A_series = A1 × A2 × …` and `A_parallel = 1 − (1 − A)^n`.

Table 12. ILLUSTRATIVE series and parallel availability, not measured.

Structure	Calculation (illustrative, each instance A = 0.999)	A
Gateway ×2	$1 - 0.001^2$	0.999999
API ×2	$1 - 0.001^2$	0.999999
PostgreSQL ×1	–	0.999
Path, improved	0.999999 × 0.999999 × 0.999	≈ 0.998998
Path, baseline (API ×1, PostgreSQL ×1, no gateway)	0.999 × 0.999	≈ 0.998001

Redundancy halves the illustrative unavailability from 0.2 % to 0.1 %, and what remains is almost entirely the single database, which agrees with the SPOF analysis of Section 6.5. Section 10.12 explains why measured results differ from this calculation.

6. System Architecture and Resilience Design

6.1 Service boundaries and data flows

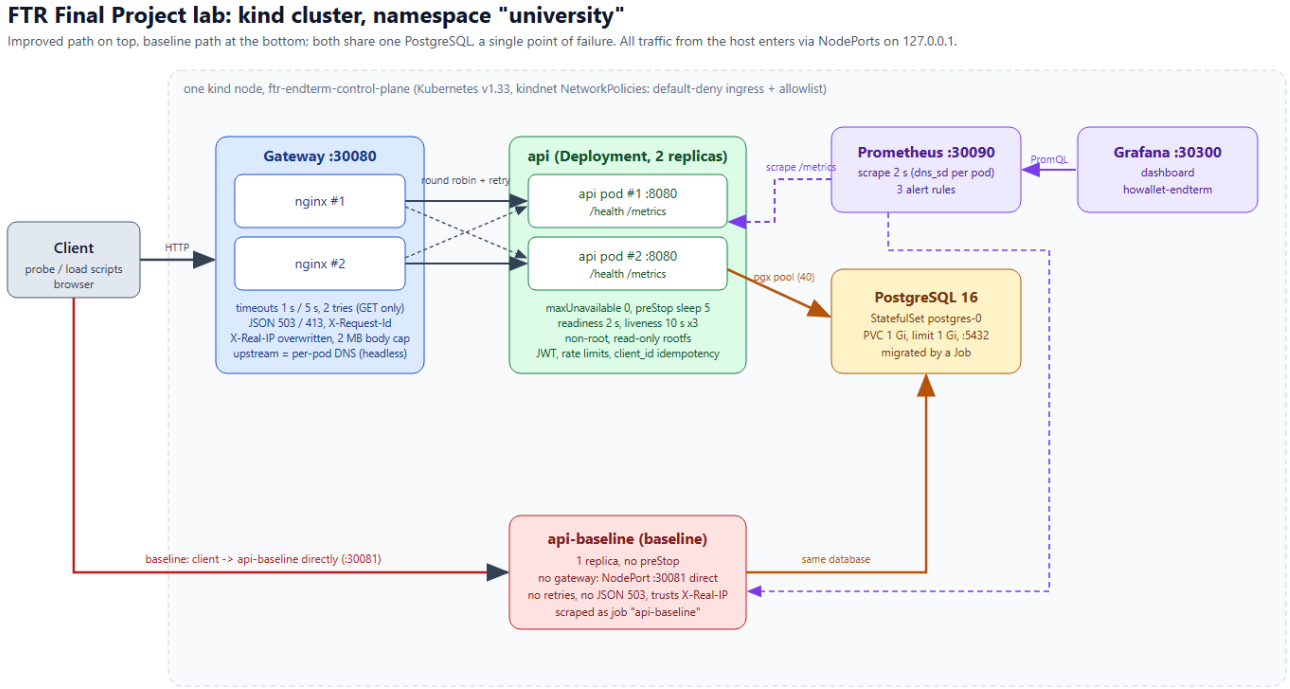


Figure 1. Lab architecture: improved path (gateway → API × 2 → PostgreSQL) with monitoring, and the baseline path at the bottom sharing the same database. Source: `scripts/architecture.html`.

AI-generated figure. The diagram was generated with the help of AI (Claude Code, as an HTML/SVG page rendered to PNG). It was carried over from the endterm, its labels were updated for this report, and it was checked by the author against the manifests in `k8s/`. The generation prompt is given in Appendix 14.6.

Table 13. Services (`notes/architecture.md`).

Service	Kind / replicas	Responsibility	Health check	Depends on	Recovery
gateway (nginx 1.27)	Deployment ×2, NodePort 30080	Single entry; timeouts, retries, JSON 503/413, correlation id, IP header overwrite	readiness and liveness `/healthz`	api, kube-dns	kubelet restart; other pod serves
api (hoWallet Go)	Deployment ×2, `maxUnavailable 0`, `maxSurge 1`	Business logic, auth, jobs, `/metrics`	readiness `/health` 2 s; liveness 10 s × 3	postgres; Frankfurter (optional)	ReplicaSet replacement; liveness restart; preStop `sleep 5`
postgres 16	StatefulSet ×1, PVC 1 Gi	Persistent store	readiness probe, period 5 s	volume	restart in place; `pg_restore` from a dump
university-migrate	Job (`backoffLimit 10`)	Applies schema migrations	–	postgres	Job retry
prometheus 2.54.1	Deployment ×1, NodePort 30090	Scrape every 2 s, 3 alert rules, 1 h retention	–	api pods	restart; history lost
grafana 11.2.0	Deployment ×1, NodePort 30300	Dashboard `howallet-endterm`	–	prometheus	dashboard provisioned from a ConfigMap
api-baseline	Deployment ×1, NodePort 30081	Comparison path	readiness only	postgres	replacement pod only

Table 14. Dependencies and failure containment.

From → to	Protocol	Required?	If it fails
client → gateway	HTTP	yes	no service
gateway → api	HTTP, per-pod DNS (`api-headless`)	yes	retry on the other pod, then JSON 503
gateway → kube-dns	DNS, `valid=2s`	for pod changes	keeps last resolved addresses; new pods not discovered
api → postgres	pgx	yes	critical functions fail; no rows half-written
api → Frankfurter	HTTPS, background job only	no	same-currency writes unaffected; foreign currency uses the archive, else `422 no_rate`
prometheus → api	HTTP `/metrics`	no	no metrics or alerts; service unaffected
grafana → prometheus	HTTP	no	no dashboard

Critical workflow, post a record. (1) The client sends `POST /api/transactions` with a bearer token, `X-Household-ID` and `client_id`. (2) The gateway sets `X-Request-Id`, overwrites `X-Real-IP` and proxies to one pod; it does not replay the POST once sent [10]. (3) API middleware runs: request id, logger, rate limit, JWT, live session, per-user limit, verified e-mail, membership. (4) `CreateOnce` looks up `client_id`; on a miss it runs one database transaction with the insert and the balance update [13]. (5) Audit, budget notification and cashback run after the commit, not atomically with it. (6) The API answers 201, or 200 for a replay.

6.2 Redundancy and backup strategy

Table 15. Redundancy and backup.

Layer	Strategy	Protects against	Does not protect against
Gateway, API	Active-active ×2, preferred anti-affinity	Loss or hang of one pod	Node loss (one node); shared DB failure
PostgreSQL	**No replica.** Restart in place, data on a PVC	Process crash	Volume loss, logical corruption
Data	Logical backup: `pg_dump -Fc`, restore with `pg_restore --clean --if-exists`	Volume loss, bad write	Writes after the last dump (RPO = time since the dump)
Backup immutability	Object-lock copy with a sha256 check was shown on the earlier lab (`ftr-a4/scripts/ransomware-drill.sh`)	Deletion of the backup	Not deployed in this lab
Release	Rolling update, `maxUnavailable 0`, `kubectl rollout undo`	Bad release	Bad migration (no automatic down-migration)

Backup is by the earlier drill only: `pg_dump` and `pg_restore` [14] were exercised in the `ftr-a3` lab (restore in 19.2 s on a very small database). No restore was run in the final project, and SLO-9 is therefore not tested.

6.3 Baseline versus improved

Both configurations run side by side, use one image and share one PostgreSQL. The application mechanisms (rollback, idempotency, rate limits, validation) exist in both, so the comparison measures infrastructure mechanisms, not the application's.

Table 16. Exactly which mechanisms differ.

Mechanism	Baseline (`api-baseline`)	Improved (`gateway` + `api`)
API replicas	1	2, `maxUnavailable 0`, `maxSurge 1`
Entry	Direct NodePort 30081	Gateway ×2, NodePort 30080
Timeouts at the edge	None; the client's timeout applies	Connect 1 s, read 5 s
Bounded retries	None	2 tries; not for a POST already sent
Passive ejection	None	`max_fails=1 fail_timeout=2s`
Failure answer	Hang, connection reset or API 500	JSON 503 with `Retry-After`
Liveness probe	**None**	`/health`, 10 s × 3
Readiness probe	`/health`, 2 s, default threshold 3	`/health`, 2 s, threshold 1, timeout 1 s
Graceful drain	None (default grace period)	preStop `sleep 5`, grace period 20 s

Mechanism	Baseline (`api-baseline`)	Improved (`gateway` + `api`)
Pod hardening	No security context in the manifest	Non-root, read-only root filesystem
Anti-affinity	None	Preferred
Client IP	Trusts client-supplied `X-Real-IP` (Exp 7)	Overwritten at the gateway
Correlation id	API id only; adopts the client's header	Gateway id, same in both logs
Body cap at the edge	None (API 1 MB decode, 16 MB ceiling)	2 MB, JSON 413
Alerts	**None of its own** (scraped as `api-baseline`)	3 rules on `job="api"`
Network policy	Own ingress policy, same DB access	Gateway and Prometheus to API only

6.4 Deployment, restart and resource assumptions

Table 17. Resource and load assumptions.

Item	Value
API	requests 50m / 64 Mi; limits 500m / 256 Mi
Gateway	requests 50m / 32 Mi; limits 300m / 128 Mi
PostgreSQL	requests 50m / 128 Mi; limits 500m / 1 Gi. At 256 Mi it was OOM-killed in the endterm and in the first run of Exp 8 (`evidence/anomalies.txt`)
DB connections	pool maximum 40 per replica; up to 80 + baseline 40 against the PostgreSQL default of 100
Reference load	30 req/s (Exp 5, one sample per run, see `results/exp5-*.json`)
Scaling	Manual; no metrics-server, so no HorizontalPodAutoscaler

6.5 Single points of failure

Table 18. Single points of failure.

Component	SPOF?	Effect when it fails	Accepted because / mitigation
PostgreSQL (1 instance, 1 PVC)	**yes**	every critical function fails (FS2); data loss if the volume is lost	lab scope; restart in place, fail-fast + JSON 503, `pg_dump` backup and restore drill (RPO = time since last dump)
kind node / host	**yes**	everything down	out of scope (assumption 1)
kube-dns	partly	gateway cannot re-resolve `api-headless` (cached `valid=2s`), new pods not discovered	kube-system replicas; nginx keeps last resolved addresses
Prometheus (1 replica)	yes, for **monitoring only**	no alerts, service itself unaffected	documented; would be a pair + Alertmanager in production
Grafana (1 replica)	yes, for dashboards only	no dashboard; alerts unaffected	–
Migration Job	start-up only	new schema not applied; the API has no schema-version check, so affected queries fail at run time	applied before the API starts; `migrate up` applies only versions above the stored one
JWT secret (one HMAC key)	yes (security)	if leaked, any token can be minted	Secret in K8s, no rotation in lab (`threat-model.md` A3)
Login lockout / rate-limit counters	not a SPOF, but **not shared**	each replica counts separately (2× limits), reset on restart	documented limitation; Exp 7: first 429 at attempt 11 instead of 6
API, gateway	**no** (2 replicas each)	–	redundancy + retries

7. Infrastructure and Software Fault-Tolerance Implementation

7.1 Infrastructure-level mechanisms

Table 19. Infrastructure mechanisms demonstrated.

Mechanism	Configuration	Demonstrated by
Instance redundancy with load balancing	API × 2 and gateway × 2 (`k8s/30-api.yaml`, `k8s/40-gateway.yaml`); per-pod upstream through the headless Service [11]	Exp 1: 418/418 with one pod deleted; 25.0–25.6 s or 2.0 s outage without it
Self-healing: restart and replacement	ReplicaSet replaces a deleted pod; liveness 10 s × 3 restarts a hung one [5]	Exp 1 recovery 1.3–1.7 s; Exp 6 hung replica back at 43.2–45.8 s
Rolling update and rollback	`maxUnavailable 0`, `maxSurge 1`, `kubectl rollout undo` [6]	Exp 8: old pods kept serving. **A default RollingUpdate with one replica did the same**, so this adds nothing measured over the default
Drain on shutdown	preStop `sleep 5`, grace period 20 s [7]	Exp 1 graceful runs: no failed request. Cost: bad pod takes 6.1–6.5 s to go in Exp 8, against 0.8–1.9 s
Database restart in place	StatefulSet, PVC; readiness `pg_isready`	Exp 2: recovers without intervention. No failover: PostgreSQL stays a SPOF

7.2 Software-level mechanisms

Six mechanisms already in the code or lab configuration are counted; none was added for this project. Go references are to the hoWallet repository at commit `06aea177`; the test output is in `analysis/ft-unit-tests.txt` (20 unit tests: **20 passed, 0 failed**).

Table 20. Software fault-tolerance mechanisms, references and tests (`notes/software-ft.md`).

#	Mechanism	Fault handled	Code / config	Unit test run	Exp
M1	Gateway timeouts and bounded retries	Dead, hung or slow API replica	`k8s/40-gateway.yaml:59-60,66-67`	none (nginx config)	3, 1
M2	Idempotent create via `client_id`	Client retry of a write whose answer was lost; concurrent duplicates	`internal/service/transaction.go:278-316`	16 passed	4, 2
M3	Database transaction rollback	Interruption between record insert and balance update	`internal/repository/postgres/postgres.go:130-174`	error propagation only; real rollback needs integration	4, 2
M4	Health and readiness checks	Crashed or hung instance receiving traffic	`k8s/30-api.yaml:64-74`, `internal/router/router.go:129-131`	none	1, 6
M5	Graceful degradation	Upstream or database down; exchange rate missing	`k8s/40-gateway.yaml:73,79-83`, `internal/service/rates.go:265-271`	2 passed	2
M6	Rate limiting with `Retry-After`	Overload, abusive client, brute force	`internal/router/router.go:115,182,209,397-398,778-788`	1 passed	5, 7

- **M1 gateway timeouts and bounded retries.** Connect 1 s, read 5 s, at most 2 tries, idempotent methods only [10]. This is the outer layer in Nygard's sense [3]; there is no circuit breaker inside the API. `max_fails=1 fail_timeout=2s` is passive ejection, and is treated as part of M1. **M1 has no Go test: it is nginx configuration, and Exp 1, 2 and 3 are its evidence.**
- **M2 idempotent create via client_id.** The lookup comes first, so a retry returns the first row with no side effects; a concurrent loser hits the unique index, rolls back and reads the winner's row. 16 unit tests ran and passed. Integration tests against PostgreSQL were cited, not run. Limit: only `POST /api/transactions` has a key.
- **M3 database transaction rollback.** `RunInTx` commits only if the function returns no error. **Its unit test TestRegister_TxError proves only that the error propagates; the integration tests were cited, not run. Exp 4 is the evidence.** Limit: audit, budget notification and cashback run after the commit.
- **M4 health and readiness checks. No Go test of the probe behaviour: Exp 1 and Exp 6 are the evidence.** Limit: `/health` does not touch the database, so readiness stays green during a database outage or a slow database link (Exp 2, Exp 3).

- **M5 graceful degradation.** JSON 503 with `Retry-After` instead of an HTML page or a hang. Limit: during a database outage the API's own JSON 500 passes through the gateway unchanged. The `no_rate` path was shown in the endterm and was not repeated here.
- **M6 rate limiting with Retry-After** (`httprate` [18]). Limit: counters live in each replica's memory (Exp 5, Exp 7).

```
upstream api_upstream {
  zone api_upstream 64k;
  server api-headless.university.svc.cluster.local:8080 resolve max_fails=1 fail_timeout=2s;   # line 35
}
location / {
  proxy_set_header X-Real-IP $remote_addr;   proxy_set_header True-Client-IP "";
  proxy_set_header X-Request-Id $req_id;     # adopted or minted correlation id
  proxy_connect_timeout 1s;  proxy_read_timeout 5s;                                          # lines 59-60
  proxy_next_upstream error timeout http_502 http_503 http_504;  proxy_next_upstream_tries 2;  # lines 66-67
  error_page 502 503 504 = @degraded;   error_page 413 = @too_large;                         # JSON bodies
}
```

Listing 1. Gateway excerpt (`k8s/40-gateway.yaml`).

7.3 Dependency-failure behaviour

If an API pod dies, the gateway retries GET on the other replica and answers JSON 503 if neither answers. If PostgreSQL stops, the membership check fails closed with a JSON 500, and requests that wait past 5 s get a JSON 503 from the gateway. Session liveness fails open on a database error, a deliberate choice to avoid mass sign-out, and a stated limitation. If the exchange-rate provider is down, base-currency writes are unaffected.

8. Monitoring, Observability and Security Implementation

8.1 Metrics and dashboard

Table 21. Metrics (metrics patch on `feat/metrics`, enabled by `METRICS_ENABLED=true`; hoWallet `main` has no metrics endpoint).

Metric	Type / labels	Used for
http_requests_total	counter {method, route, status}	request rate, error-rate alert
http_request_duration_seconds	histogram {method, route}	p50, p95, p99; latency alert
http_requests_in_flight	gauge	saturation
db_pool_acquired_conns, db_pool_total_conns	gauges from pgxpool	pool starvation
up (Prometheus)	per scraped pod	ApiReplicaDown alert

Prometheus scrapes every 2 s and evaluates rules every 2 s, discovering API pods per replica through DNS on the headless Service. The Grafana dashboard `howallet-endterm` (dashboard name kept from the endterm) shows only `job="api"`, so baseline numbers come from the result files.



Figure 2. Grafana during the Exp 6 hung-replica series: one instance's up falls to 0, the firing table shows ApiReplicaDown, the error rate panel stays at 0 %.



Figure 3. Grafana during Exp 2 rep 1 (database stopped): p95 and p99 reach the 5–10 s bound, 500 and 499 responses appear, ApiHighLatencyP95 fires. Replicas stay up, since `/health` does not touch the database.

8.2 Structured logs and request correlation

Both tiers log one JSON object per request. nginx adopts an incoming `X-Request-Id` or mints one and forwards it; the API adopts it, writes it to its log line with `user_id` and `household_id`, and persists the same lines to the `app_logs` table. Samples below are from `evidence/log-samples.txt`, shortened; identifiers are synthetic.

```
# Exp 3 improved: slow replica cut at 5 s, retried on the healthy one; client sees 200
gateway {"request_id":"exp3-improved-rep3-25","method":"GET","uri":"/api/accounts","status":200,
         "upstream_addr":"10.244.0.7:8080, 10.244.0.8:8080","request_time":5.005,"upstream_status":"504, 200"}
api     {"level":"ERROR","msg":"request","status":500,"duration":5000730056,"request_id":"exp3-improved-rep3-25"}
api     {"level":"INFO","msg":"request","status":200,"duration":2626313,"request_id":"exp3-improved-rep3-25"}
# Exp 4 improved: interrupted POST is not replayed; the client's retry with the same client_id succeeds
gateway {"request_id":"exp4-improved-rep3-try1","method":"POST","status":503,"upstream_status":"502"}
gateway {"request_id":"exp4-improved-rep3-try2","method":"POST","status":201,"upstream_status":"201"}
```

```
# Exp 7: denied request, one id in both logs
gateway {"request_id":"exp7a-r2-05","method":"GET","uri":"/api/accounts","status":404,"upstream_status":"404"}
api     {"level":"INFO","msg":"request","status":404,"request_id":"exp7a-r2-05","user_id":"9fe8558a-…"}
```

Listing 2. Correlated log lines, shortened from `evidence/log-samples.txt`.

8.3 Alert rules and incident indicators

Table 22. Alert rules (`k8s/50-prometheus.yaml`) [15] and observed firing delay from the injection (Exp 6).

Alert / severity	Expression (summary)	for	Indicates	Firing delay measured
ApiHighErrorRate / critical	5xx rate / all rate > 0.05 over 30 s, job=api, route≠/health	10 s	server-side failures	14.2–16.2 s (database stopped)
ApiHighLatencyP95 / warning	histogram p95 over 30 s > 0.5 s	10 s	slow responses, locks, saturation	16.2 s (lock bursts)
ApiReplicaDown / critical	(count(up{job=api} == 1) < 2) or absent(up{job=api} == 1)	6 s	fewer than 2 healthy replicas	11.2–12.2 s (hung replica)

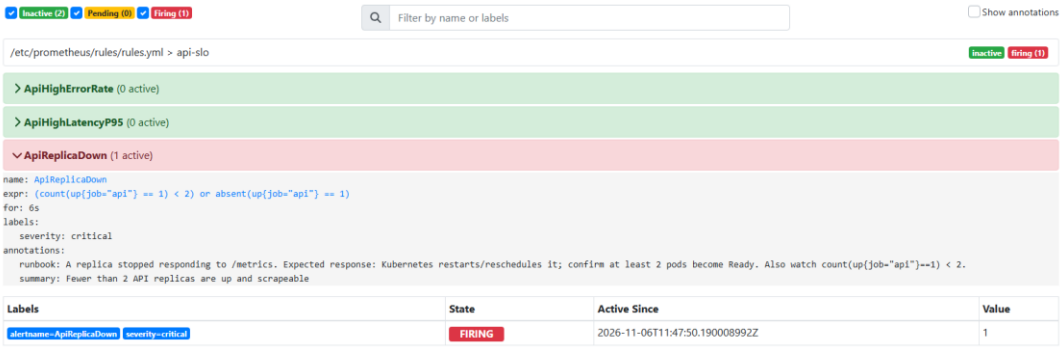


Figure 4. Prometheus alert page: ApiReplicaDown firing in Exp 6 rep 1 (screenshot taken by `AlertWatcher` during the run).

How an incident is diagnosed. An alert goes pending, then firing. The dashboard separates the cases: error rate up with replicas up points to a dependency; replicas below 2 points to a pod; latency up with no errors points to contention. The `request_id` from the client or the gateway log then finds the API line, and `upstream_status` shows the retries. **Gap measured in Exp 8: no alert watches rollouts**, so a stuck rollout is invisible to monitoring.

8.4 Security controls

Table 23. Security controls (`notes/security-controls.md`).

#	Control	Type	Where	Section	Tested in
C1	JWT authentication (HMAC only, issuer and expiry required, live-session check)	**Authentication**	`internal/middleware/auth.go:90-160`, `internal/middleware/session.go:125-140`	(a)	Exp 7 MU1
C2	Household membership check, 404 for non-members	**Access control**	`internal/middleware/auth.go:180-217`	(b)	Exp 7 MU2
C3	Rate limits and login lockout	Abuse containment	`internal/router/router.go:115,182,209`, `internal/service/auth.go:40-109`	(c), (d)	Exp 7 MU4, Exp 5
C4	Input validation and body limits	Input safety	`internal/handler/helpers.go:129-131`, `k8s/40-gateway.yaml` (2 MB cap)	(e)	Exp 7 MU3
C5	Request and audit logging with `request_id`	Detection, audit	`internal/middleware/logger.go:42-101`, `internal/service/audit.go`	(g)	Exp 7
C6	NetworkPolicies, non-root, read-only root filesystem	Containment	`k8s/70-network-policy.yaml`, `k8s/30-api.yaml`	threat model, A3	not tested in Exp 7

Controls map to OWASP API Security Top 10 2023 risks [17], mainly API1 (object level authorisation), API2 (authentication) and API4 (resource consumption). Secrets in the manifests are synthetic lab placeholders. Known weaknesses are stated: limits and lockout are per replica, session liveness fails open on a database error, there is no dedicated failed-login event, and the API's database user owns every table. NetworkPolicies [8] and pod hardening were deployed but not tested in Exp 7.

9. Verification, Validation and Experimental Methodology

9.1 Environment and configurations

One-node kind cluster [9] `ftr-endterm` (name kept from the endterm) on Docker Desktop, Windows 11 laptop, WSL2, namespace `university`. All traffic comes from one Python client on the same machine, through `127.0.0.1` NodePorts. Valid runs are from 6 November 2026, about 16:08–17:08 local time. The hypotheses, workloads, durations and criteria in `evidence/test-plan.md` were written before the runs. The configurations differ as in Table 16.

9.2 Tooling and repetitions

- `scripts/run_exp.py <1..8>` runs all repetitions of one experiment and appends the record to `evidence/expN.txt`; raw data lands in `evidence/results/expN-*.json`. `bash run_all.sh` runs all eight.
- `scripts/ftlab.py`: an open-loop probe (a hanging request does not slow it), an endpoint watcher and an alert watcher polling Prometheus every second. Each run first waits for all workloads ready and no alert active.
- `scripts/faults.py` (Exp 1, 2), `scripts/final.py` (Exp 3, 4, 8), `scripts/exps.py` (Exp 5, 6, 7), `scripts/build_summary.py` (consolidated table and reliability inputs). `tc netem` [12] injects the delay.

9.3 Test plan

Table 24. Experiment matrix as run (`evidence/test-plan.md`). B = baseline, I = improved.

Exp	Question	Fault	Workload	Reps	Hypothesis written before the run	Success criteria
1	Does another instance keep the workflow available?	Delete one API pod: graceful, graceful, forced	GET /api/accounts, open loop, 8 req/s	3 per config	I: at most a few failed requests per pod loss. B: outage until the replacement is ready	I availability ≥ 99 % after injection, ≥ 99.9 % for graceful. B: no target
2	Fail safely, recover, no corruption?	Postgres scaled to 0 (2 runs) or SIGSTOP (2 runs), 20 s	GET at 6 req/s per config, timeout 12 s, plus one create per 1.5 s retried with same client_id	4, both configs at once	Both fail and recover with no lost or duplicate write. I within about 10 s, B waits to the 12 s client timeout. An alert fires (I)	One row per acknowledged write, balance = −Σ, recovery ≤ 30 s, I errors JSON, alert fires
3	Are timeouts and retries bounded and effective?	`tc netem delay 2000ms` to port 5432 in one API pod, 40 s	GET at 2 req/s, client timeout 60 s, plus one create per 1.5 s	3 per config	I: cut at 5 s and retried once, ≥ 99 %, max ≤ 12 s; readiness will not remove the replica. **B: every request about 8–10 s**	I: delayed packets seen, ≥ 99 %, max ≤ 12 s, ≤ 2 tries, writes intact, latency normal ≤ 30 s after removal. B: no target
4	Recover without losing or duplicating a record?	SIGKILL the API process while its create waits between insert and balance update (row lock held by a test session)	2 creates, the interrupted create retried every 0.5 s; GET at 4 req/s; background creates	3 per config	Rollback leaves nothing, retry gives one row in both. I retry in about 1–2 s; B only after the restart	1 row for the id, 3 rows on the account, balance = −Σ, no duplicate anywhere, retry ≤ 60 s (I ≤ 10 s)
5	How do throughput, latency and errors change?	None: load 5 → 30 → 5 req/s, 30 + 60 + 30 s	GET, 5 users, timeout 6 s; 429 counted apart	2 per config	I serves all with p95 < 500 ms. **B also holds this rate**, so the difference may be small	≥ 99 % excluding 429, p95 < 500 ms in the step, no alert
6	Detected and diagnosed from telemetry?	Hung replica (SIGSTOP); database stopped 25 s; 22 lock bursts over about 40 s; quiet window	GET at 6 req/s (quiet 5 req/s); Prometheus polled every 1 s	3, 2, 2, 2 (I only)	Each fault fires its alert within about 45 s; none in the quiet window; other alerts may also fire	Expected alert firing ≤ 45 s, clears after recovery, zero in quiet
7	Access control and validation enforced and logged?	None: 11 access-control cases, input cases, 12 wrong passwords	Sequential requests; 5 req/s probe during input cases	2 per set	401, 404, 400/413, 429; data unchanged; each request has both log lines; X-Real-IP spoof not honoured through the gateway	Each case returns its expected status; md5 snapshot unchanged; all logged
8	Return to an acceptable state?	`set env API_PORT=9999`, 45 s, then `rollout undo`	GET at 5 req/s, timeout 12 s, plus one create per 1.5 s	2 per config	I: no failed request. **B: probably the same.** Monitoring will not notice the stuck rollout	≥ 99 %, bad pod never ready, template restored, recovery ≤ 60 s, writes intact

Recorded per run: fault, start and end, detection time, recovery time, successful and failed requests, latency p50, p95 and max, retries and failovers, data-integrity result. Each run records whether the fault was really in place. Client retries reuse the same `client_id`. Integrity is checked in SQL: rows against acknowledged writes, distinct `client_id`, `balance = −Σ amounts`, and no duplicate (`household_id, client_id`) anywhere.

9.4 Automated tests

Table 25. Tests relevant to this project.

Level	Count	Status in this project
Unit tests of the fault-tolerance mechanisms	20	**Run** on 6 Nov 2026 at 06aea177: 20 passed (`analysis/ft-unit-tests.txt`)
All Go unit tests	2095 top-level	Run on 4 Nov 2026 at 06aea177: 0 failed (`notes/test-inventory.md`)
Integration tests on real PostgreSQL	278 (static count)	**Cited, not run**: Docker was busy with the lab
API tests over a real app and database	538 (static count)	**Cited, not run**

9.5 Invalid runs and amendments

Negative findings about the lab are kept in the record. `evidence/anomalies.txt` and the two dated amendments in `evidence/test-plan.md` are the sources.

Table 26. Runs set aside, why, and what was done.

Run	Why	Action
`exp4-*-rep1-INVALID-kill-not-effective` (2)	SIGKILL had no effect: restart count unchanged, first POST answered 201	Kept, excluded, rerun
`exp8-improved-rep1-INVALID-confounded-postgres-oom`	Postgres OOM-killed at its 256 Mi limit (exit 137, peak 268 MB, 68 connections); availability 3.61 % in the fault window. It measured the memory limit, not the rollout	Kept, excluded. Dated environment change to 1 Gi, for both configs. All later runs share one environment
`results/pre-pg-memory-fix/` (Exp 3, Exp 4 rep 1)	Valid measurements of a different environment (256 Mi)	Set aside, not used; rerun
`results/superseded/` (Exp 1 × 6, Exp 6 latency × 2)	Script defects: Exp 1 stopped too early; the alert watcher could stop just before a clear	Set aside; stop conditions fixed before the repeats. No hypothesis or criterion changed

Finding: PostgreSQL at 256 Mi. After the limit went to 1 Gi, the same broken release ran at 100 % availability. That supports memory pressure as the cause but does not prove it: it rests on one run pair. Two further readings limit the claim. At 1 Gi, the Exp 5 improved runs show a Postgres working set of 247.7–252.6 MiB, close to the old limit, and working set includes reclaimable cache. A `memory.peak` of 210 MB recorded at 17:01 covers only the time since Postgres last restarted (Exp 6 stopped it), so it says nothing about the Exp 8 runs. The endterm also saw a Postgres OOM kill, under normal lab load.

10. Results and Baseline-versus-Improved Comparison

Every value here is measured, on 6 November 2026, unless labelled otherwise. Ranges are min–max across valid repetitions. With 2–4 repetitions they describe behaviour and are not statistical estimates.

10.1 Consolidated results table

Table 27 on the next page is `evidence/results/summary.md`, reproduced by script when this report is built. Experiment 6 has no baseline row because the baseline has no alert rules of its own.

Table 27. Consolidated results, baseline versus improved (`evidence/results/summary.md`, every value measured).

Exp	Config	Valid reps	Fault injected	Detection (s)	Recovery (s)	OK / total	p50 ms	p95 ms	max ms	Retries / failovers	Data integrity	Result
1	improved	3	pod delete: 2 graceful, 1 forced	0.2-0.3 endpoint drop	1.3-1.7; outage 0.0	418/418 (100.0 %)	9.4-9.8	47.4-151.2	329.5-926.0	1 retried, 1 failovers OK, 0 degraded 503, max 2 tries	n/a (read-only probe)	PASS
1	baseline	3	pod delete: 2 graceful, 1 forced	0.2 endpoint drop	2.3-25.9; outage 2.0-25.6	538/840 (64.048 %)	9.0-4056.7	1244.9-6016.2	2145.5-6039.6	0 (no gateway)	n/a (read-only probe)	comparison, no target
2	improved	4	postgres: 2 stop, 2 SIGSTOP freeze, 20 s	0.3-2.8 first failure; alerts firing 15.2-21.2	4.0-12.3 after restore	40/522 (7.663 %) during outage	8.1-5005.9	5012.1-9179.7	10010.5-10015.8	188 retried, 12 failovers OK, 569 degraded 503, max 2 tries	146 acked, 146 rows, 0 dup; balances OK	PASS
2	baseline	4	postgres: 2 stop, 2 SIGSTOP freeze, 20 s	0.3-2.8 first failure	3.2-8.0 after restore	182/489 (37.219 %) during outage	11677.0-12002.8	12013.4-12014.4	12020.2-12031.8	0 (no gateway)	143 acked, 143 rows, 0 dup; balances OK	PASS
3	improved	3	netem 2000 ms api->postgres, 40 s	1.0-1.5 first slow	1.42-9.84 after removal	247/247 (100.0 %) during fault	10.0-10.7	5010.1-5013.4	5028.9-5083.2	31 retried, 31 failovers OK, 0 degraded 503, max 2 tries; client write retries 18	165 acked, 165 rows, 0 dup; balances OK	PASS
3	baseline	3	netem 2000 ms api->postgres, 40 s	1.0 first slow	4.28-4.48 after removal	173/247 (70.04 %) during fault	10011.0-18035.5	18641.6-32048.1	22371.9-34030.4	0 (no gateway); client write retries 1001	165 acked, 165 rows, 0 dup; balances OK	comparison, no target; writes OK
4	improved	3	SIGKILL between insert and balance move	0.2 client error	1.2-1.7 to successful retry	376/376 (100.0 %) probe after kill	9.2-9.6	26.5-138.8	117.0-1024.8	3 retried, 3 failovers OK, 3 degraded 503, max 2 tries; client tries 2	interrupted id rows 1; balance == -sum all; DB dup 0	PASS
4	baseline	3	SIGKILL between insert and balance move	0.2 client error	3.6-34.1 to successful retry	277/400 (69.25 %) probe after kill	8.4-10355.6	26.8-12018.6	929.8-12028.5	0 (no gateway); client tries 5-7	interrupted id rows 1; balance == -sum all; DB dup 0	PASS
5	improved	2	load step 5->30->5 req/s	n/a	n/a	3600/3600 (100.0 %) in step phase	9.2-9.6	25.7-28.5	88.7-165.7	0 retried, 0 failovers OK, 0 degraded 503, max 1 tries	n/a (read-only)	PASS
5	baseline	2	load step 5->30->5 req/s	n/a	n/a	2479/2520 (98.373 %), 1080 429 in step phase	8.4	769.7-1094.8	2034.2-2292.1	0 (no gateway)	n/a (read-only)	FAIL
6	improved	3	hung replica (SIGSTOP)	ApiReplicaDown firing 11.2-12.2	alert cleared 52.6-55.5	1042/1076 (96.84 %)	9.6-9.8	31.9-89.3	5009.2-5028.3	0 retried, 0 failovers OK, 34 degraded 503, max 1 tries	also fired: none	PASS
6	improved	2	database stopped 25 s	ApiHighErrorRate firing 14.2-16.2	alert cleared 53.4-54.5	9/303 (2.97 %)	6.5-9.2	5009.2-5018.7	10007.7-10024.7	89 retried, 2 failovers OK, 276 degraded 503, max 2 tries	also fired: ApiHighLatencyP95	PASS
6	improved	2	lock bursts about 40 s	ApiHighLatencyP95 firing 16.2	alert cleared 69.8-70.7	480/480 (100.0 %)	599.9-610.1	1411.6-1421.0	1510.7-1519.3	0 retried, 0 failovers OK, 0 degraded 503, max 1 tries	also fired: none	PASS
6	improved	2	none: quiet 120 s	n/a	n/a	1200/1200 (100.0 %)	9.4-9.5	23.3-27.4	78.9-94.0	-	alerts seen: 0	PASS
7	gateway	2	access control	22/22 logged in gateway+API	n/a	20/22 cases as expected	-	-	-	-	victim snapshot unchanged: yes	FAIL
7	gateway	2	input and limits	38/38 gateway, 36/38 API; first 429 at attempt 11	n/a	38/38 cases as expected	-	-	-	-	accounts before/after: ['1', '1'], ['1', '1']	PASS
8	improved	2	broken release API_PORT=9999, then rollout undo	2.2-2.4 readiness failure; alerts: none	6.11-6.49 after undo	530/530 (100.0 %)	9.4-9.9	51.5-101.6	196.2-1019.5	0 retried, 0 failovers OK, 0 degraded 503, max 1 tries	104 acked, 104 rows; config restored: yes	PASS
8	baseline	2	broken release API_PORT=9999, then rollout undo	2.1-2.4 readiness failure; alerts: none	0.76-1.94 after undo	479/479 (100.0 %)	8.5-8.7	26.1-26.4	139.0-213.9	0 (no gateway)	98 acked, 98 rows; config restored: yes	PASS

"comparison, no target" means the plan set no target for that row. Baseline has no alert rules of its own, so Exp 6 has no baseline row. Exp 7 FAIL: see Section 10.9. Exp 5 baseline FAIL: see Section 10.7.

10.2 Required metrics by experiment

Table 28. Availability, failure rate, recovery and detection. R_req and A are over the whole window; the consolidated table gives the fault window.

Exp	Config	R_req	A	Observed failure rate, basis	MTTR	Detection (s)	Recovery (s)
1	baseline	68.51 %	55.95 %	90 per hour, 3 in 119.5 s	17.6 s	0.2 endpoint drop	2.3-25.9; outage 2.0-25.6
1	improved	100.00 %	100.00 %	0 per hour, 0 in 66.7 s	undefined	0.2-0.3 endpoint drop	1.3-1.7; outage 0.0
2	baseline	75.28 %	50.42 %	70 per hour, 4 in 207.0 s	25.7 s	0.3-2.8 first failure	3.2-8.0 after restore
2	improved	50.56 %	44.95 %	69 per hour, 4 in 207.7 s	28.6 s	0.3-2.8 first failure; alerts firing 15.2-21.2	4.0-12.3 after restore
3	baseline	86.27 %	62.13 %	40 per hour, 3 in 268.0 s	33.8 s	1.0 first slow	4.28-4.48 after removal
3	improved	100.00 %	100.00 %	0 per hour, 0 in 247.5 s	undefined	1.0-1.5 first slow	1.42-9.84 after removal
4	baseline	74.85 %	54.94 %	89 per hour, 3 in 121.5 s	18.3 s	0.2 client error	3.6-34.1 to successful retry
4	improved	100.00 %	100.00 %	0 per hour, 0 in 115.6 s	undefined	0.2 client error	1.2-1.7 to successful retry
5	baseline	98.69 %	97.42 %	90 per hour, 6 in 240.4 s	1.0 s	n/a	n/a
5	improved	100.00 %	100.00 %	0 per hour, 0 in 240.4 s	undefined	n/a	n/a
6	improved	91.39 %	88.72 %	25 per hour, 5 in 720.2 s	16.2 s	ApiReplicaDown firing 11.2-12.2	alert cleared 52.6-55.5
8	baseline	100.00 %	100.00 %	0 per hour, 0 in 146.8 s	undefined	2.1-2.4 readiness failure; alerts: none	0.76-1.94 after undo
8	improved	100.00 %	100.00 %	0 per hour, 0 in 156.0 s	undefined	2.2-2.4 readiness failure; alerts: none	6.11-6.49 after undo

10.3 Exp 1: application instance failure

Table 29. Exp 1 per run, after injection, probe 8 req/s. Latency in ms, times in s.

Config	Rep	OK / total	p50	p95	max	Endpoint drop	Endpoints back	Recovery	Outage
baseline	1 graceful	181/330	4056.7	6015.3	6030.6	0.2	0.9	25.9	25.6
baseline	2 graceful	187/331	3581.7	6016.2	6039.6	0.2	1.1	25.3	25.0
baseline	3 forced	170/179	9.0	1244.9	2145.5	0.2	1.3	2.3	2.0
improved	1 graceful	140/140	9.8	47.4	349.8	0.3	1.3	1.3	0.0
improved	2 graceful	138/138	9.4	151.2	926.0	0.2	1.4	1.4	0.0
improved	3 forced	140/140	9.7	54.2	329.5	0.2	1.7	1.7	0.0

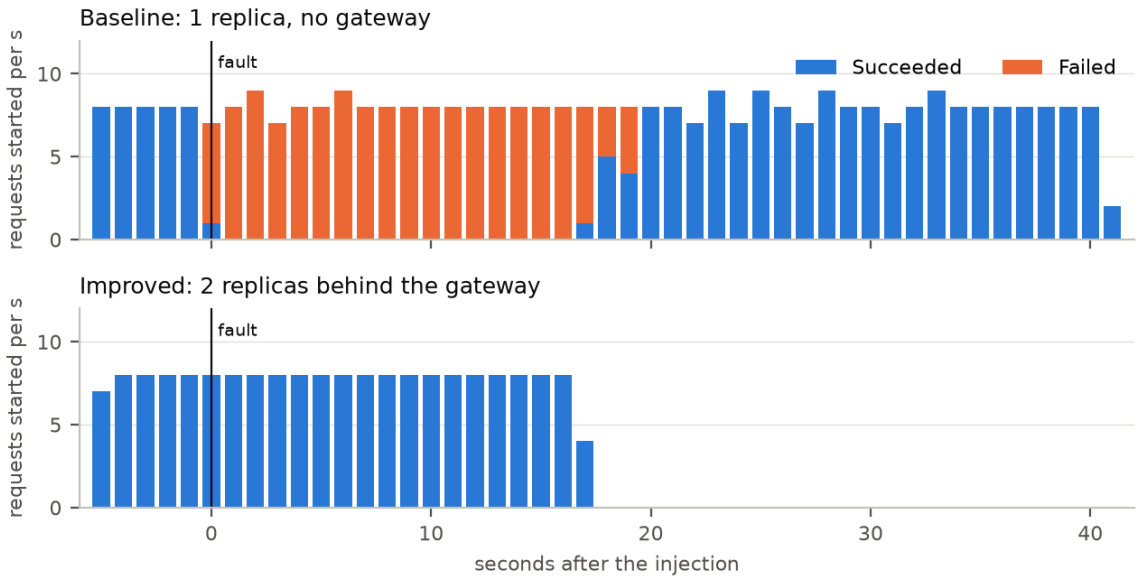


Figure 5. Exp 1 rep 1 (graceful delete): requests per second by outcome, from the raw records. The baseline fails for about 25 s although its endpoint returned at 0.9 s; the improved configuration shows no failed request.

Hypothesis held for improved; baseline worse than the plan expected. The improved configuration failed no request in 3 runs; one request failed over in the gateway log (`502, 200`). The baseline's graceful delete caused a 25.0–25.6 s outage although endpoints returned in 0.9–1.1 s; the forced delete caused 2.0 s. **Why a graceful delete takes longer than a forced one is not established.** The plan expected an outage until the replacement was ready; the replacement was ready far sooner than the outage ended, and readiness on `/health` does not show that the pod can serve database requests. The endterm's explanation, pool starvation during start-up, was not tested here.

Correction. The endterm's 23 s was a lower bound, because that probe stopped while requests were still timing out. The run now records `clients_settled_before_stop`, true in all six runs, and measures 25.0–25.6 s.

10.4 Exp 2: database unavailable

Table 30. Exp 2 per run, during the outage. Latency in ms; recovery in s after the restore command; writes = acknowledged / rows; alert firing in s.

Run	Config	OK / total	p50	p95	max	Codes	Recovery	Writes	Alerts firing
rep1-stop	baseline	24/130	12002.8	12014.2	12031.8	200:24, 500:15, timeout:91	8.0	34/34	no rule
rep1-stop	improved	5/138	20.0	5012.1	10015.8	200:5, 500:19, 503:114	4.0	34/34	HighErrorRate 15.2, HighLatencyP95 15.2
rep2-stop	baseline	45/114	11677.0	12013.4	12029.9	200:45, 500:16, timeout:53	3.2	38/38	no rule
rep2-stop	improved	1/123	8.8	5025.8	10012.7	200:1, 500:21, 503:101	12.3	39/39	HighErrorRate 15.2, HighLatencyP95 15.2
rep3-freeze	baseline	62/130	11999.7	12013.8	12020.2	200:62, timeout:66, 500:2	4.3	37/37	no rule
rep3-freeze	improved	17/138	8.1	5021.8	10010.5	200:17, 503:116, 500:5	7.4	38/38	HighErrorRate 21.2, HighLatencyP95 21.2
rep4-freeze	baseline	51/115	12001.8	12014.4	12028.6	200:51, timeout:64	3.9	34/34	no rule
rep4-freeze	improved	17/123	5005.9	9179.7	10011.0	200:17, 503:102, 500:4	7.3	35/35	HighErrorRate 20.2, HighLatencyP95 20.2

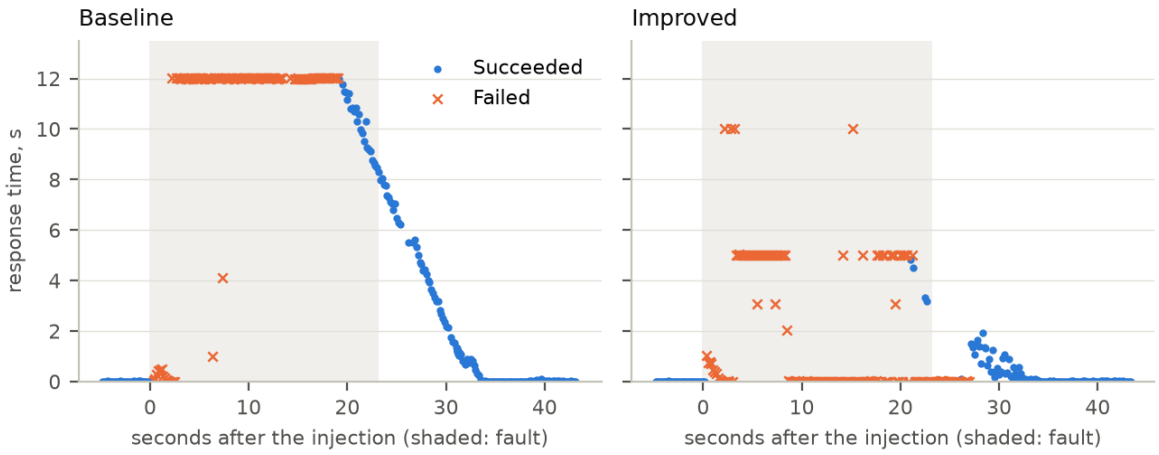


Figure 6. Exp 2 rep 1 (stop): response time of every GET, from the raw records. The improved configuration's failures end within 5–10 s or at once; the baseline's wait up to the 12 s client timeout.

The success ratio favours the baseline: 37 % vs 8 % during the outage, 75.28 % vs 50.56 % over the whole window. This must not be read as an availability gain for improved, and it is not a real gain for the baseline. No configuration can serve these reads with its only database down. The baseline's requests wait up to the 12 s client timeout, so some finish after the restore and count as successes. The gateway gives up within 10 s, and after one timeout `max_fails=1` marks both replicas down, so later requests get a fast 503 (`502` from `no live upstreams`). **The improved gain is bounded response time: maximum 10.0 s and a machine-readable reply with Retry-After, against 12.0 s at the client's own limit.** The price is fewer late successes and slower recovery after the restore (4.0–12.3 s vs 3.2–8.0 s). 188 requests were retried, only 12 of those succeeded on the other replica, and 569 got JSON

503. Both configurations recovered without intervention within the 30 s criterion, and all 146 and 143 acknowledged writes have exactly one row. Both alerts fired at 15.2–21.2 s, and correctly: requests really were slow and failing.

10.5 Exp 3: network delay or timeout (new)

Table 31. Exp 3 per run, during the fault. Latency in ms. Gateway retries all succeeded (`504, 200`). Client write retries are the writer's. Last slow = end of the last request over 200 ms after removal, s.

Config	Rep	Delayed packets	OK / total	p50	p95	max	Gateway retries	Client write retries	Last slow	Alerts firing (s)
baseline	1	988	31/82	18035.5	32048.1	34030.4	–	287	1.1	no rule
baseline	2	1075	75/83	10011.0	18641.6	22371.9	–	393	0.5	no rule
baseline	3	1066	67/82	10011.3	19686.4	25528.4	–	321	0.7	no rule
improved	1	308	82/82	10.0	5013.4	5028.9	11	5	0.0	HighErrorRate 22.2, HighLatencyP95 22.2
improved	2	285	82/82	10.1	5010.1	5083.2	8	8	9.8	HighErrorRate 36.5, HighLatencyP95 24.4
improved	3	319	83/83	10.7	5011.2	5037.8	12	5	0.0	HighErrorRate 32.3, HighLatencyP95 23.2

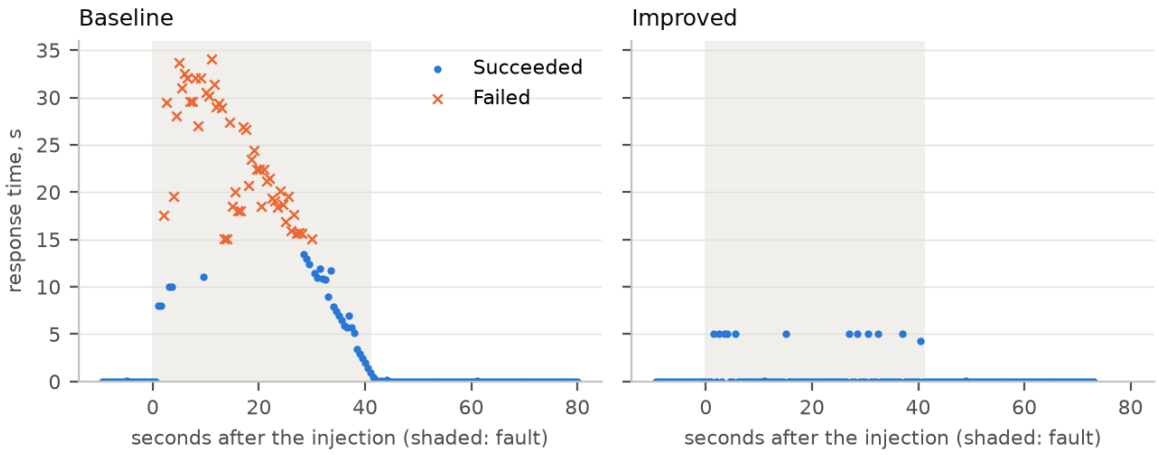


Figure 7. Exp 3 rep 1: response time of every GET. Improved: requests on the slow replica are cut at 5 s and retried, the rest stay near 10 ms. Baseline: nothing bounds the wait until connections are closed.

Baseline hypothesis failed. The plan predicted 8–10 s per request. Measured p50 was 10.0–18.0 s, maximum 34 s, and 30 % failed (74 of 247), as connections closed by the server without a response. A likely reason is the API's 15 s `WriteTimeout`, but **this is a hypothesis, not tested**. A confounder is the client's 1001 write retries, mostly 429 from the 60 creates/min route limit, which add load on the baseline (18 on improved). **Improved passed:** 247/247, maximum 5.08 s against the 12 s bound, at most 2 tries, 31 retries all `504, 200`. Readiness did not remove the slow replica, as predicted, because `/health` does not touch the database. The 9.84 s recovery in one run comes from one 235 ms request after the removal, just over the 200 ms threshold; the others give 3.78 s and 1.42 s.

False-positive trade-off. ApiHighErrorRate fired in all three improved runs, though **clients saw no failure**: the API records a request the gateway timed out as a 500 (Listing 2 shows one, with the client's 200 beside it). ApiHighLatencyP95 fired too, correctly. In two runs the error alert was pending, cleared and returned before firing.

10.6 Exp 4: interrupted multi-step transaction (new)

Table 32. Exp 4 per run. `Insert` done and `Committed` are read from `pg_locks` and the table at the moment of the kill.

Config	Rep	Insert done	Committed	Container restarts	Client tries	Retry OK (s)	Probe OK / total	Rows for id	Rows on account	Balance / expected	DB dup	Alerts firing (s)
baseline	1	True	0	5 -> 6	5	3.6	119/127	1	3/3	-306 / -306	0	no rule
baseline	2	True	0	6 -> 7	6	22.2	93/127	1	3/3	-306 / -306	0	no rule
baseline	3	True	0	7 -> 8	7	34.1	65/146	1	3/3	-306 / -306	0	no rule

Config	Rep	Insert done	Committed	Container restarts	Client tries	Retry OK (s)	Probe OK / total	Rows for id	Rows on account	Balance / expected	DB dup	Alerts firing (s)
improved	1	True	0	2 -> 3	2	1.7	127/127	1	3/3	-306 / -306	0	none
improved	2	True	0	3 -> 4	2	1.3	122/122	1	3/3	-306 / -306	0	ReplicaDown 10.6
improved	3	True	0	4 -> 5	2	1.2	127/127	1	3/3	-306 / -306	0	ReplicaDown 9.6

Hypothesis held on integrity, and it does not separate the configurations. Idempotency and the transaction live in the application, so both have them. Each process was killed with the insert done and nothing committed; Postgres logged `unexpected EOF on client connection with an open transaction`; each retry was answered 201, so the first attempt really was rolled back. **The difference is retry time and probes:** improved retries succeed in 1.2–1.7 s through the other replica, and its probe stays at 376/376. The gateway does not replay the POST; the client's retry does the recovery. **Baseline retry time grew 3.6 → 22.2 → 34.1 s with restart count.** Kills were about 44 s apart, which likely puts the container into kubelet's restart back-off [7], so **3.6 s is the fair figure for a first crash**. The back-off also shows on improved (endpoints back at 1.5, 19.0 and 31.3 s), where the other replica hides it; ApiReplicaDown fired in two of those runs, correctly.

10.7 Exp 5: high load

Table 33. Exp 5, 60 s step at 30 req/s (1800 requests per run). Latency in ms. CPU in millicores per API pod, one sample per run.

Config	Rep	200	429	Conn. errors	OK req/s	p50	p95	max	API CPU	Alerts
baseline	1	1264	516	20	21.07	8.4	769.7	2034.2	61.2	none
baseline	2	1215	564	21	20.25	8.4	1094.8	2292.1	61.6	none
improved	1	1800	0	0	30.0	9.2	25.7	88.7	34.3, 29.2	none
improved	2	1800	0	0	30.0	9.6	28.5	165.7	38.1, 37.3	none

The plan's hypothesis that the baseline also holds this rate failed, and the failure is the limiter, not capacity. API CPU stayed near 61.2–61.6 millicores. 1080 of 3600 requests got 429 once the 1200/min per-process limit ran out, 36–39 s into the step, which is why the baseline row fails (98.37 % excluding 429, p95 769.7–1094.8 ms). **The improved configuration avoids this only because each replica counts separately (900/min each): its limit is effectively doubled, not shared. The improvement is therefore partly an artefact of the limiter**, and this experiment does not show that two replicas add capacity that was needed. **The cause of the 41 connection errors is not established**; they appear only while the limiter is active. At low load the gateway costs little: warm-phase p50 is 9.9 ms against 9.1–9.3 ms.

10.8 Exp 6: monitoring and alert trigger

Table 34. Exp 6 per run. Latency in ms; alert times in s after the injection.

Run	Expected alert	OK / total	p95	max	Alerts
quiet-window-rep1	none	600/600	27.4	94.0	none seen
quiet-window-rep2	none	600/600	23.3	78.9	none seen
replica-down-rep1	ApiReplicaDown	339/350	89.3	5028.3	ReplicaDown: firing 11.2, clear 53.0
replica-down-rep2	ApiReplicaDown	356/369	71.4	5009.2	ReplicaDown: firing 11.2, clear 55.5
replica-down-rep3	ApiReplicaDown	347/357	31.9	5027.8	ReplicaDown: firing 12.2, clear 52.6
error-rate-rep1	ApiHighErrorRate	6/152	5009.2	10007.7	HighErrorRate: firing 16.2, clear 53.4; HighLatencyP95: firing 22.2, clear 57.5
error-rate-rep2	ApiHighErrorRate	3/151	5018.7	10024.7	HighErrorRate: firing 14.2, clear 54.5; HighLatencyP95: firing 20.2, clear not recorded
latency-rep1	ApiHighLatencyP95	240/240	1421.0	1510.7	HighLatencyP95: firing 16.2, clear 69.8
latency-rep2	ApiHighLatencyP95	240/240	1411.6	1519.3	HighLatencyP95: firing 16.2, clear 70.7

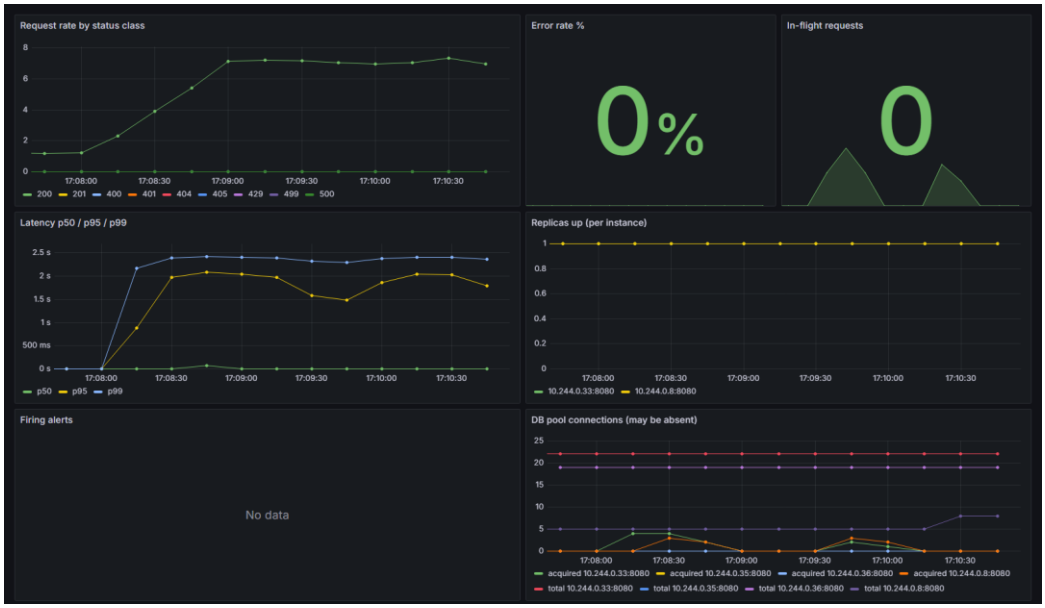


Figure 8. Grafana during the Exp 6 lock bursts: latency rises with no failed request, ApiHighLatencyP95 fires.

Hypothesis held. Each fault fired its alert within 45 s (11.2–16.2 s) and cleared after recovery; no alert fired in either quiet window. A database outage also fires ApiHighLatencyP95, which is correct: requests really were slow. In error-rate rep 2 that secondary alert's clear time was not recorded; the primary's was.

Hung replica: not all requests failed over. 34 of 1076 requests got a 503 after 5 s, with a single try (`504`) and no failover, all within about 5 s of the fault. Exp 3's slow replica did fail over. **A hypothesis, not tested,** is that readiness removed the hung replica from DNS while those requests were in flight, leaving nginx no other peer for them. The replica left endpoints at 2.4–2.7 s and returned at 43.2–45.8 s after the liveness restart.

10.9 Exp 7: unauthorized or invalid operation

Table 35. Exp 7 access control, rep 1. Rep 2 gave the same statuses.

Case	Status	Expected	As expected
no token GET /api/accounts	401	401	yes
no token POST /api/transactions	401	401	yes
garbage token GET /api/households	401	401	yes
token signed with wrong secret	401	401	yes
B reads A's accounts (X-Household-ID=A)	404	404	yes
B reads A's transactions	404	404	yes
B creates expense on A's account	404	404	yes
B deletes A's account	404	404	yes
B uses own household but A's account id	400	4xx refusal	yes
B GET /api/households/{A}	405	404	**no**
A reads own accounts (control)	200	200	yes

Recorded result: FAIL, 20 of 22 cases as expected. The 2 failed cases are one case in both repetitions: `B GET /api/households/{A}` expected 404 and got **405**, because that route has no GET. Nothing was disclosed: the body is empty. The case tests routing, not tenant isolation. **The expected value was wrong, but it stays as written before the run, and the result is not re-scored.** The other cases show tenant isolation enforced: 401 for missing, garbage or forged tokens, 404 for all of user B's attempts on A's record space, and a 400 refusal for A's account id used from B's own household. The md5 snapshot of the victim's data is unchanged, and 22/22 requests are in both logs.

Table 36. Exp 7 input and limit cases, rep 1 (rep 2 the same). 38/38 cases as expected over both reps.

Case	Status	Retry-After
malformed JSON POST /api/accounts	400	–
malformed JSON POST /api/transactions	400	–
negative amount	400	–
invalid UUID in path	400	–
3 MB body via gateway	413	–
1.5 MB body via gateway (API's own 1 MB decode limit)	400	–
correct password after the burst	429	900

Twelve wrong passwords gave `[401, 401, 401, 401, 401, 401, 401, 401, 401, 401, 429, 429]`: **first 429 at attempt 11, not 6**, because each replica counts its own 5 failures (a weakness the threat model predicted). The correct password afterwards still got 429 with `Retry-After: 900`. Background probes stayed OK (39/39 and 40/40). The 3 MB bodies stop at the gateway with JSON 413, which is why those 2 requests are missing from the API log (36/38 in the API log, 38/38 at the gateway). **Finding:** on the baseline the API logged `remote=203.0.113.77`, an address chosen by the client, and therefore a fresh limiter and lockout key per request; through the gateway it logged `10.244.0.1`. Failed logins have no dedicated audit event, only the request line.

10.10 Exp 8: recovery and rollback (new)

Table 37. Exp 8 per run. Latency in ms; readiness failure in s after the release; bad pod gone in s after the undo.

Config	Rep	OK / total	p50	p95	max	Readiness failure	Bad pod gone	Writes	DB dup	Restored	Alerts
baseline	1	237/237	8.5	26.1	213.9	2.4	0.76	49/49	0	yes	none
baseline	2	242/242	8.7	26.4	139.0	2.1	1.94	49/49	0	yes	none
improved	1	265/265	9.9	51.5	196.2	2.4	6.11	52/52	0	yes	none
improved	2	265/265	9.4	101.6	1019.5	2.2	6.49	52/52	0	yes	none

Both pass, and this experiment shows no gain from the improved configuration, as the plan predicted: a default RollingUpdate with 1 replica rounds `maxUnavailable` down to 0 and `maxSurge` up to 1 [6], so it keeps the old pod. The improved bad pod takes longer to go (6.11–6.49 s vs 0.76–1.94 s) because of the preStop `sleep 5`. **No alert noticed the stuck rollout in any run: a monitoring gap.** Separately, the first run of the same broken release at the 256 Mi Postgres limit was a real outage (3.61 % availability), kept as evidence that the database is a single point of failure (Section 9.5).

10.11 Data consistency, retries and objectives

Table 38. Data consistency across all write runs: no acknowledged write lost or duplicated in any valid run of either config.

Exp	Config	Runs	Acknowledged	Rows	Duplicates in database	Balance = −Σ
2	baseline	4	143	143	0	yes
2	improved	4	146	146	0	yes
3	baseline	3	165	165	0	yes
3	improved	3	165	165	0	yes
4	baseline	3	90	90	0	yes
4	improved	3	86	86	0	yes
8	baseline	2	98	98	0	yes
8	improved	2	104	104	0	yes

Table 39. Measured outcome per service-level objective (improved configuration).

SLO	Target	Measured	Outcome
SLO-1 steady availability	≥ 99.5 %	1200/1200 in quiet windows (Exp 6)	Met

SLO	Target	Measured	Outcome
SLO-2 during instance loss	≥ 99 %	418/418 (Exp 1)	Met
SLO-3 latency at 30 req/s	p95 ≤ 500 ms	25.7–28.5 ms (Exp 5)	Met, with the limiter caveat
SLO-4 bounded failure latency	GET ≤ 10.5 s; POST ≤ 5.5 s	GET max 10.016 s (Exp 2), 5.08 s (Exp 3). POST was not separated in the results	GET met; POST not measured separately
SLO-5 recovery	≤ 10 s instance; ≤ 30 s database	1.3–1.7 s (Exp 1); 4.0–12.3 s (Exp 2); 6.11–6.49 s (Exp 8)	Met
SLO-6 write integrity	0 lost, 0 duplicates, balance check	All runs (table above)	Met; application mechanism, present in both configs
SLO-7 detection	≤ 30 s, no false positive in quiet	11.2–16.2 s (Exp 6); 0 alerts in quiet	Met; but a false positive under fault in Exp 3, and a rollout gap in Exp 8
SLO-8 access control	100 % denied, 0 rows changed, logged	Isolation enforced, snapshot unchanged, 22/22 logged. Exp 7 recorded FAIL on one mis-specified case	Recorded FAIL; see 10.9
SLO-9 backup recoverability	0 records lost since dump	No restore run in this project	**Not tested**

Table 40. Hypotheses against results.

Exp	Hypothesis	Result
1	Improved: at most a few failures. Baseline: outage until the replacement is ready	Improved held. Baseline outage outlasted readiness by far; cause not established
2	Both fail and recover with no lost write; improved bounded, baseline waits; alert fires	Held. Success ratio favours the baseline; explained in 10.4
3	Improved bounded and retried; baseline 8–10 s per request	Improved held. **Baseline failed**: p50 10–18 s, 30 % failed
4	One row after retry in both; improved retry 1–2 s; baseline after restart	Held; integrity does not separate the configs
5	Improved p95 < 500 ms; baseline also holds the rate	Improved held. **Baseline part failed**, by the limiter
6	Each fault fires its alert ≤ 45 s; none in quiet	Held; 34 of 1076 got 503 with no failover
7	Expected statuses; data unchanged; logged; spoof not honoured through the gateway	Held except the expected value of one case: **recorded FAIL**
8	No failed request; baseline probably the same; monitoring will not notice	Held in all three parts: no gain from improved

10.12 Measured results against theory

Section 5.4 is illustrative; everything else is measured. The formula assumes independent failures and instant failover, and the measurements show otherwise. Both API replicas fail together when the shared database is lost (Exp 2), so redundancy gives nothing there. A replacement pod can be Ready before it can serve (Exp 1 baseline). Passive ejection likely delays recovery after a database restore (4.0–12.3 s vs 3.2–8.0 s). A request in flight on a hung replica may get no failover (Exp 6). The measured A values, such as 55.95 % or 44.95 %, cannot be compared with the illustrative 0.999: they describe short windows built around a deliberate fault, and the illustrative figure describes a long-run process that was not observed.

10.13 Trade-offs

Table 41. Trade-offs of the improved configuration, measured unless marked.

Trade-off	Evidence
Fewer late successes during a dependency outage	Exp 2: 8 % vs 37 % during the outage; response bounded at 10.0 s
Slower recovery after a database restore	Exp 2: 4.0–12.3 s vs 3.2–8.0 s
False-positive alert	Exp 3: ApiHighErrorRate fired in 3 of 3 runs while clients saw no failure
Slower shutdown	Exp 8: bad pod gone in 6.11–6.49 s vs 0.76–1.94 s (preStop)
Added latency	Exp 5 warm phase p50: 9.9 ms vs 9.1–9.3 ms. Retried requests cost about 5 s (Exp 3)
Resources	Configured limits: 2 API + 2 gateways, 1600 m CPU and 768 Mi, against 500 m and 256 Mi (from the manifests). Sampled in the Exp 5 step: API 29.2–38.1 m per pod, gateway 10.9–13.9 m per pod, baseline API 61.2–61.6 m

Trade-off	Evidence
Complexity	Gateway config with timeouts, retries, per-pod DNS; rules can be wrong, as the endterm's ApiReplicaDown was
Limiter behaviour changes with replica count	Exp 5, Exp 7: effective limits double, lockout needs more attempts

11. Discussion, Limitations and Future Work

11.1 What the evidence supports

Redundancy with a retrying gateway turns the loss of one instance from an outage into no visible failure (Exp 1), and gateway timeouts bound the damage of one slow replica (Exp 3). Alerts detect a hung replica, a lost database and contention in 11–16 s, and stay silent in quiet windows (Exp 6). Correct records survived every injected fault in both configurations, thanks to the application's transaction and idempotency key, which are not improvements of this project's configuration. The clearest mechanism-level value is in Exp 1, Exp 3 and the retry time of Exp 4.

11.2 What the evidence does not support

- **Availability during a database outage.** Improved is lower on success ratio; the gain is bounded response time.
- **Capacity from two replicas.** Exp 5 is confounded by a per-process limiter.
- **Rollback safety from the improved configuration.** Exp 8 shows no gain over the Kubernetes default.
- **Failure rates.** MTTF, MTBF and λ reflect injection timing.
- **Causes marked as hypotheses or not established:** the graceful-delete outage in Exp 1, the `WriteTimeout` in Exp 3, the 41 connection errors in Exp 5, the DNS explanation in Exp 6, kubelet back-off in Exp 4, and memory pressure as the cause of the Postgres OOM run.

11.3 Limitations

- **One node, one laptop.** All replicas share one host, and the load generator runs on it. Pod-level faults cannot show node failure, and **replicas on one node do not protect against node failure.**
- **PostgreSQL remains a single point of failure.** There is no replica; only restart in place and an earlier manual backup drill. SLO-9 was not tested.
- **Small repetition counts** (2–4 per scenario): ranges describe behaviour, not statistics, and no confidence intervals are claimed. Exp 3 runs differ widely even within a config, and Exp 4 baseline runs affect each other through restart back-off.
- **Shared environment.** Both configurations share one database, and baseline and improved pods stay deployed during each other's runs. Exp 2 probes both at once, which is fair for the fault but puts both loads on one recovering database.
- **Test defects kept as they are:** one wrong expected value (Exp 7) and a bad `kill` in the first Exp 4 runs.
- **Tests cited, not run:** integration and API tests. M1 and M4 have no Go test.
- **Reproduction needs the images.** The API image is built from hoWallet at `06aea177` plus the metrics patch; the application source and the patch are private, available on request (Appendix 14.4).

11.4 Future work

- A database replica with failover, or at least scheduled backups with a timed restore drill, to address the SPOF and SLO-9.
- A separate startup gate or database-aware readiness, and an experiment that finds the cause of the graceful-delete outage.
- Shared rate-limit and lockout counters, then a repeat of Exp 5 with the limiter removed from the comparison.
- A `statement_timeout` and a pool-acquire timeout, so a blocked database fails inside the API.

- An alert on stuck rollouts, and an error-rate alert computed at the gateway, to remove the Exp 3 false positive.
- More repetitions on several nodes, with a load generator on another machine.

12. Conclusion

The dependability objectives were met for instance failure, bounded response under delay, write integrity, detection and recovery time, with the caveats given per objective. SLO-9 was not tested, and Exp 7 stands as a recorded FAIL for a wrong expected value. The improved configuration clearly helps when one instance fails or slows down. It does not make a database outage more available; it changes how that outage looks to the client. It gains nothing measurable in the rollback experiment, its load result is partly a limiter artefact, and it adds a false-positive alert. The main residual risk is the single PostgreSQL instance, and the lab's single node limits every availability claim to pod-level faults.

13. References

[1] A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr, "Basic Concepts and Taxonomy of Dependable and Secure Computing," *IEEE Transactions on Dependable and Secure Computing*, vol. 1, no. 1, pp. 11–33, 2004. doi:10.1109/TDSC.2004.2

[2] B. Beyer, C. Jones, J. Petoff, N. R. Murphy (eds.), *Site Reliability Engineering: How Google Runs Production Systems*, O'Reilly, 2016. Ch. 4 "Service Level Objectives", ch. 6 "Monitoring Distributed Systems", ch. 21 "Handling Overload". https://sre.google/sre-book/table-of-contents/

[3] M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed., Pragmatic Bookshelf, 2018. Stability patterns: timeouts, circuit breakers, bulkheads.

[4] J. A. Hanley, A. Lippman-Hand, "If Nothing Goes Wrong, Is Everything All Right? Interpreting Zero Numerators," *JAMA*, vol. 249, no. 13, pp. 1743–1745, 1983. doi:10.1001/jama.1983.03330370053031 (rule of three).

[5] Kubernetes documentation, "Liveness, Readiness, and Startup Probes." https://kubernetes.io/docs/concepts/configuration/liveness-readiness-startup-probes/

[6] Kubernetes documentation, "Deployments" (rolling update, `maxSurge` rounded up, `maxUnavailable` rounded down, rollback). https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

[7] Kubernetes documentation, "Pod Lifecycle" (termination, preStop hooks, container restart back-off of 10 s, 20 s, 40 s, capped at 300 s). https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/

[8] Kubernetes documentation, "Network Policies." https://kubernetes.io/docs/concepts/services-networking/network-policies/

[9] kind documentation, "Quick Start." https://kind.sigs.k8s.io/docs/user/quick-start/

[10] nginx documentation, "Module ngx_http_proxy_module" (`proxy_next_upstream`, `proxy_next_upstream_tries`, `proxy_connect_timeout`, `proxy_read_timeout`). https://nginx.org/en/docs/http/ngx_http_proxy_module.html

[11] nginx documentation, "Module ngx_http_upstream_module" (`server … resolve`, open source since 1.27.3, `max_fails`, `fail_timeout`). https://nginx.org/en/docs/http/ngx_http_upstream_module.html

[12] Linux manual page, "tc-netem(8)." https://man7.org/linux/man-pages/man8/tc-netem.8.html

[13] PostgreSQL 16 documentation, "3.4. Transactions." https://www.postgresql.org/docs/16/tutorial-transactions.html

[14] PostgreSQL 16 documentation, "pg_dump." https://www.postgresql.org/docs/16/app-pgdump.html

[15] Prometheus documentation, "Alerting rules." https://prometheus.io/docs/prometheus/latest/configuration/alerting_rules/

[16] A. Shostack, *Threat Modeling: Designing for Security*, Wiley, 2014. STRIDE.

[17] OWASP, "OWASP Top 10 API Security Risks – 2023." https://api-security.owasp.org/editions/2023/en/0x11-t10/

[18] `go-chi/chi` v5.3.0 and `go-chi/httprate` v0.15.0 (HTTP router, rate limiter), `jackc/pgx` v5.9.2 (PostgreSQL driver), `shopspring/decimal` v1.4.0 (money arithmetic). https://github.com/go-chi/chi, https://github.com/go-chi/httprate, https://github.com/jackc/pgx, https://github.com/shopspring/decimal. Versions: `go.mod` at commit `06aea177`.

14. Appendix: Code Repository, Configuration, Test Scripts, Logs and Supplementary Tables

14.1 Repository layout

```
ftr-final/
  README.md             reproduction steps, ports, gateway behaviour
  run_all.sh            one command: all eight experiments, then the summary
  kind-config.yaml, Dockerfile.migrate
  k8s/                  00-namespace, 10-postgres, 20-migrate-job, 30-api, 40-gateway, 50-prometheus, 60-grafana,
                        70-network-policy; baseline/api-baseline.yaml; optional/block-egress.yaml
  scripts/              run_exp.py, faults.py, final.py, exps.py, ftlab.py, build_summary.py, log_samples.py, demo.py,
                        setup_users.py, up.sh, down.sh, architecture.html, grafana_shot.py
  analysis/             reliability.py, reliability-computed.json, ft-unit-tests.txt
  notes/                requirements, reliability, architecture, software-ft, threat-model, security-controls,
                        graceful-degradation, test-inventory, references
  evidence/             test-plan.md, anomalies.txt, log-samples.txt, exp1..8.txt, results/, screenshots/, endterm-
results/
  report/               this report, slides, demo-runbook.md, build_report.py, build_deck.js, fig/
```

The repository is a local git repository. `evidence/token.txt` and `evidence/lab-users.json` are git-ignored. Secrets in `k8s/00-namespace.yaml` and the password in `scripts/ftlab.py` are **synthetic lab placeholders**, and the JWT in `scripts/exps.py` is the deliberately forged test token of Exp 7 (subject all zeros, signed with a wrong secret). No real credential is included.

14.2 One-command reproduction

```
bash scripts/up.sh --baseline     # create the cluster, load images, apply manifests, wait for Ready
python scripts/setup_users.py     # synthetic users; tokens go to the git-ignored evidence/lab-users.json (valid
24 h)
bash run_all.sh                   # all eight experiments, about 1 hour, then evidence/results/summary.md
python analysis/reliability.py --json-out analysis/reliability-computed.json   # reliability table
python report/build_report.py     # this report (needs Word for the PDF; --no-pdf for DOCX only)
bash scripts/down.sh              # delete the cluster
```

A single experiment: `python scripts/run_exp.py <1..8> [rep ...]`. Rerunning a repetition overwrites its JSON.

14.3 Raw data locations

Table 42. Where each kind of evidence lives.

What	Where
Hypotheses, criteria, amendments	`evidence/test-plan.md`
Readable record per experiment	`evidence/exp1.txt` … `exp8.txt`
Raw per-request records and per-run results	`evidence/results/exp*.json`
Consolidated table and reading notes	`evidence/results/summary.md`
Inputs and computed reliability values	`evidence/results/reliability-inputs.json`, `analysis/reliability-computed.json`
Excluded and set-aside runs	`*-INVALID-*.json`, `results/pre-pg-memory-fix/`, `results/superseded/`, `evidence/anomalies.txt`
Log samples with correlation ids	`evidence/log-samples.txt`
Dashboards and firing alerts	`evidence/screenshots/`
Unit test output for the mechanisms	`analysis/ft-unit-tests.txt`
Endterm data, for reference only	`evidence/endterm-results/`

14.4 Building the images

`scripts/up.sh` needs local images `howallet-api:endterm` and `ftr-migrate:endterm`. The API image is built from the hoWallet repository at commit `06aea177` with the opt-in `/metrics` patch applied (commit `60cd5791` on `feat/metrics`), using its `deployments/Dockerfile.api`. The application source and the patch are private and not in this repository;

they are available to the instructor on request. Without them, the instructor can inspect the configuration, scripts and raw data, and the demonstration runs on the author's machine.

14.5 Supplementary: reliability per run

Table 43. Reliability quantities per repetition (`python analysis/reliability.py --per-run`).

Exp	Config	Run	T_obs (s)	N_f	D (s)	MTTR (s)	A	ok / failed	R_req
1	baseline	exp1-baseline-rep1-graceful	46.1	1	25.6	25.6	44.44 %	221 / 149	59.73 %
1	baseline	exp1-baseline-rep2-graceful	46.1	1	25.0	25.0	45.78 %	226 / 144	61.08 %
1	baseline	exp1-baseline-rep3-crash	27.3	1	2.0	2.0	92.63 %	210 / 9	95.89 %
1	improved	exp1-improved-rep1-graceful	22.3	0	0.0	–	100.00 %	179 / 0	100.00 %
1	improved	exp1-improved-rep2-graceful	22.1	0	0.0	–	100.00 %	178 / 0	100.00 %
1	improved	exp1-improved-rep3-crash	22.3	0	0.0	–	100.00 %	179 / 0	100.00 %
2	baseline	exp2-rep1-stop	48.0	1	30.7	30.7	36.10 %	183 / 106	63.32 %
2	baseline	exp2-rep2-stop	55.5	1	23.4	23.4	57.90 %	265 / 69	79.34 %
2	baseline	exp2-rep3-freeze	54.0	1	24.5	24.5	54.58 %	257 / 68	79.08 %
2	baseline	exp2-rep4-freeze	49.5	1	24.1	24.1	51.38 %	233 / 65	78.19 %
2	improved	exp2-rep1-stop	48.2	1	26.7	26.7	44.65 %	136 / 154	46.90 %
2	improved	exp2-rep2-stop	55.7	1	32.5	32.5	41.61 %	171 / 164	51.04 %
2	improved	exp2-rep3-freeze	54.2	1	27.7	27.7	48.93 %	164 / 162	50.31 %
2	improved	exp2-rep4-freeze	49.7	1	27.5	27.5	44.65 %	161 / 138	53.85 %
3	baseline	exp3-baseline-rep1	89.5	1	43.1	43.1	51.88 %	129 / 51	71.67 %
3	baseline	exp3-baseline-rep2	93.0	1	22.9	22.9	75.39 %	179 / 8	95.72 %
3	baseline	exp3-baseline-rep3	85.5	1	35.5	35.5	58.44 %	157 / 15	91.28 %
3	improved	exp3-improved-rep1	82.5	0	0.0	–	100.00 %	166 / 0	100.00 %
3	improved	exp3-improved-rep2	82.5	0	0.0	–	100.00 %	166 / 0	100.00 %
3	improved	exp3-improved-rep3	82.5	0	0.0	–	100.00 %	166 / 0	100.00 %
4	baseline	exp4-baseline-rep1	39.0	1	1.8	1.8	95.49 %	149 / 8	94.90 %
4	baseline	exp4-baseline-rep2	39.0	1	20.2	20.2	48.09 %	123 / 34	78.34 %
4	baseline	exp4-baseline-rep3	43.5	1	32.8	32.8	24.73 %	94 / 81	53.71 %
4	improved	exp4-improved-rep1	39.0	0	0.0	–	100.00 %	157 / 0	100.00 %
4	improved	exp4-improved-rep2	37.5	0	0.0	–	100.00 %	151 / 0	100.00 %
4	improved	exp4-improved-rep3	39.1	0	0.0	–	100.00 %	157 / 0	100.00 %
5	baseline	exp5-baseline-rep1	120.2	3	2.6	0.9	97.83 %	1560 / 20	98.73 %
5	baseline	exp5-baseline-rep2	120.2	3	3.6	1.2	97.01 %	1517 / 21	98.63 %
5	improved	exp5-improved-rep1	120.2	0	0.0	–	100.00 %	2102 / 0	100.00 %
5	improved	exp5-improved-rep2	120.2	0	0.0	–	100.00 %	2102 / 0	100.00 %
6	improved	exp6-error-rate-rep1	66.2	1	27.5	27.5	58.43 %	240 / 158	60.30 %
6	improved	exp6-error-rate-rep2	64.2	1	27.2	27.2	57.65 %	226 / 160	58.55 %
6	improved	exp6-latency-rep1	78.2	0	0.0	–	100.00 %	470 / 0	100.00 %
6	improved	exp6-latency-rep2	78.2	0	0.0	–	100.00 %	470 / 0	100.00 %
6	improved	exp6-quiet-window-rep1	120.0	0	0.0	–	100.00 %	600 / 0	100.00 %
6	improved	exp6-quiet-window-rep2	120.0	0	0.0	–	100.00 %	600 / 0	100.00 %
6	improved	exp6-replica-down-rep1	63.2	1	9.7	9.7	84.69 %	369 / 11	97.11 %
6	improved	exp6-replica-down-rep2	66.2	1	8.8	8.8	86.64 %	385 / 13	96.73 %
6	improved	exp6-replica-down-rep3	64.2	1	8.0	8.0	87.52 %	376 / 10	97.41 %
8	baseline	exp8-baseline-rep1	73.4	0	0.0	–	100.00 %	368 / 0	100.00 %
8	baseline	exp8-baseline-rep2	73.4	0	0.0	–	100.00 %	368 / 0	100.00 %
8	improved	exp8-improved-rep1	78.0	0	0.0	–	100.00 %	391 / 0	100.00 %
8	improved	exp8-improved-rep2	78.0	0	0.0	–	100.00 %	391 / 0	100.00 %

14.6 AI-generated architecture diagram: prompt

The architecture diagram (Section 6.1) was generated by an AI coding assistant (Claude Code) inside a longer working session, so the exact wording of the original request was not preserved. The prompt below reproduces it: given to the assistant together with the `k8s/` manifests, it yields `scripts/architecture.html`, which is then rendered to `report/fig/architecture.png`. All facts on the diagram were checked by the author against the manifests.

```
Read the Kubernetes manifests in k8s/ (namespace, postgres, migrate job, api, gateway, prometheus,
grafana, network policies) and k8s/baseline/api-baseline.yaml. Draw the architecture of this lab as a
single self-contained HTML page with an inline SVG (1400 x 760, white background, Segoe UI), so it can
be rendered to PNG for a report. Use only facts from the manifests.

Show:
- a dashed frame for the single kind node and the namespace "university", noting the default-deny
  ingress NetworkPolicy with an allowlist;
- the Client on the left (probe/load scripts, browser);
- the improved path on top: Gateway (NodePort 30080) with two nginx pods, then the api Deployment with
  two pods (:8080, /health, /metrics), then PostgreSQL 16 (StatefulSet postgres-0, PVC, :5432,
  migrated by a Job);
- under each box, its resilience settings in small text: gateway timeouts and retries (GET only),
  JSON 503/413, X-Request-Id, X-Real-IP overwritten, body cap, per-pod DNS upstream; api replicas,
  maxUnavailable 0, preStop, readiness and liveness probes, non-root and read-only rootfs, JWT, rate
  limits, client_id idempotency;
- monitoring on the right: Prometheus (NodePort 30090, scrape interval, dns_sd per pod, 3 alert
  rules) and Grafana (NodePort 30300, dashboard), with arrows "scrape /metrics" and "PromQL";
- the baseline path at the bottom: api-baseline, 1 replica, no preStop, no gateway, NodePort 30081
  direct, no retries, no JSON 503, trusts X-Real-IP, scraped as job "api-baseline";
- that both paths share the same PostgreSQL, which is a single point of failure.

Label the arrows (HTTP, round robin + retry, pgx pool, "same database"). Use one colour per tier
(client grey, gateway blue, api green, database amber, monitoring violet, baseline red), a title on top and a
one-line subtitle explaining the two paths. Keep the text readable when the image is 17 cm wide.
```